

基于智能体技术的 Web 无障碍检测系统设计与实现

摘要

本文围绕 Web 无障碍自动化检测中页面遍历策略僵化、动态内容适应能力不足、语义层面问题难以发现等问题，设计并实现了一套基于智能体技术的 Web 无障碍检测系统 W4Agent。系统以 WCAG 2.1 和 GB/T 37668-2019 等标准为依据，采用前后端分离架构，后端基于 FastAPI 构建服务接口，结合 Playwright 完成浏览器自动化执行，利用 LangGraph/LangChain 组织多智能体协同流程，前端基于 React 与 Ant Design 构建任务管理、实时监控、报告查看和人工标注工作台。

系统核心包括多智能体编排引擎、自适应智能遍历器、多层次无障碍检测引擎和人机协同前端工作台。多智能体编排引擎通过 Master Agent、Crawler Agent、Detector Agent 和 Reporter Agent 分工协作，实现检测任务的规划、探索、检测与报告生成；自适应智能遍历器结合启发式 URL 优先级评分、DOM 结构签名去重和突发事件响应机制，提高动态网站场景下的页面覆盖能力；检测引擎在传统规则检测基础上引入大语言模型语义分析和多模态视觉分析，以发现规则难以覆盖的语义一致性和视觉可访问性问题。最后，系统通过可视化报告和截图标注功能，为测试人员提供直观的问题定位和修复依据。

本文完成了 W4Agent 系统的需求分析、总体设计、核心模块实现和测试方案设计。实验部分将从功能完整性、页面覆盖率、检测准确性、误报率、检测耗时等方面对系统进行评估，并与主流无障碍检测工具进行对比。结果表明，基于智能体技术的检测方法能够在复杂动态 Web 场景中提升检测流程的灵活性和问题发现能力，为 Web 无障碍自动化测试提供了一种可行的工程实现方案。

关键词： Web 无障碍；智能体；大语言模型；自适应遍历；WCAG；多模态检测

Abstract

This thesis focuses on the limitations of existing automated Web accessibility testing methods, including rigid crawling strategies, insufficient adaptability to dynamic Web pages, and limited capability in detecting semantic accessibility issues. To address these problems, this thesis designs and implements W4Agent, an intelligent Web accessibility detection system based on agent technology. The system follows WCAG 2.1 and GB/T 37668-2019 as the main accessibility criteria. It adopts a front-end and back-end separated architecture: the back end is built with FastAPI, browser automation is implemented with Playwright, multi-agent orchestration is organized with LangGraph and LangChain, and the front end is developed with React and Ant Design to provide task management, real-time monitoring, report viewing, and human annotation functions.

The core modules of W4Agent include a multi-agent orchestration engine, an adaptive crawler, a multi-layer accessibility detection engine, and a human-in-the-loop workspace. The multi-agent engine coordinates Master Agent, Crawler Agent, Detector Agent, and Reporter Agent to complete task planning, page exploration, accessibility detection, and report generation. The adaptive crawler improves page coverage in dynamic Web environments by combining heuristic URL prioritization, DOM-structure-based deduplication, and unexpected event handling. The detection engine integrates rule-based checks, LLM-based semantic analysis, and multimodal visual analysis, which helps identify issues that are difficult to detect using deterministic rules alone. In addition, the system provides visual screenshot annotations and structured reports to help testers locate and fix accessibility issues efficiently.

This thesis presents the requirement analysis, system design, module implementation, and testing plan of W4Agent. The evaluation is designed from the perspectives of functional completeness, page coverage, detection accuracy, false positive rate, execution time, and comparison with mainstream accessibility testing tools. The system demonstrates that an agent-based approach can improve the flexibility and issue discovery capability of automated Web accessibility testing in complex dynamic Web scenarios.

Key Words: Web Accessibility; Agent; Large Language Model; Adaptive Crawling; WCAG; Multimodal Detection

目录

- 第 1 章 绪论
- 第 2 章 相关技术与理论基础
- 第 3 章 系统需求分析
- 第 4 章 系统总体设计
- 第 5 章 核心模块设计与实现
- 第 6 章 系统测试与实验分析
- 第 7 章 总结与展望
- 参考文献
- 致谢
- 附录

第 1 章 绪论

1.1 研究背景

图片占位：图 1-1 Web 无障碍检测应用背景示意图

建议内容：展示普通用户、老年用户、视障用户、辅助技术、Web 服务之间的关系，可自绘。

随着互联网技术的持续发展，Web 应用已经成为社会公共服务、商业活动、在线教育、医疗健康、政务办理和日常生活的重要入口。大量信息资源和业务流程从线下迁移到线上，用户通过浏览器即可完成信息检索、表单填写、商品购买、预约挂号、在线缴费等操作。在这一背景下，Web 页面不仅是信息展示媒介，也逐渐成为社会参与和公共服务获得的重要基础设施。对于老年人、视障人士、听障人士、肢体障碍人士以及存在临时性使用障碍的用户而言，如果网站在页面结构、交互控件、文本替代、键盘操作、颜色对比度等方面缺乏合理设计，就会直接影响其获取信息和完成操作的能力。

Web 无障碍的目标是使不同能力、不同设备和不同使用场景下的用户都能够平等、有效地访问网络内容。无障碍设计并不仅服务于特定残障群体，也能够提升网站整体可用性。例如，清晰的页面标题和结构化标题层级有助于搜索引擎和辅助技术理解页面内容；图片替代文本能够帮助屏幕阅读器用户获取图像信息，也能在图片加载失败时提供必要说明；良好的颜色对比度不仅方便低视力用户阅读，也能提升移动端强光环境下的可读性；键盘可操作性不仅服务于无法使用鼠标的用户，也能提升熟练用户的操作效率。因此，Web 无障碍既是信息公平的重要体现，也是现代 Web 应用质量评价的重要维度。

国际上，万维网联盟 W3C 发布的 Web Content Accessibility Guidelines（WCAG）已经成为 Web 无障碍设计和检测的重要标准。WCAG 以“可感知、可操作、可理解、健壮性”四大原则为核心，对文本替代、时间媒体、适应性布局、键盘访问、导航、输入辅助、兼容性等方面提出了系统要求。国内也发布了 GB/T 37668-2019《信息技术 互联网内容无障碍可访问性技术要求与测试方法》等标准，并通过相关政策和法律推动无障碍

环境建设。随着无障碍环境建设逐渐从倡导性要求走向制度化要求，如何对网站进行高效、准确、可持续的无障碍检测，成为网站开发、测试、运维和合规评估中的重要问题。

传统 Web 无障碍检测主要依赖人工检查和规则驱动的自动化工具。人工检查能够较好地理解页面语义和用户体验，但成本高、效率低，并且受测试人员经验影响较大。自动化检测工具则能够快速扫描页面 DOM 结构、属性和样式，发现图片缺少替代文本、表单缺少标签、页面缺少语言声明、标题层级异常等确定性问题。然而，随着现代 Web 应用逐渐呈现出单页应用化、组件化、动态加载和强交互等特征，单纯依靠静态规则扫描已经难以满足复杂检测需求。一方面，许多页面内容需要用户点击、滚动、填写或切换路由后才能出现，传统爬虫或线性扫描工具难以覆盖关键交互路径；另一方面，Cookie 横幅、登录弹窗、广告遮挡、浏览器 Alert 等突发事件可能中断检测流程；此外，许多无障碍问题并非简单 DOM 规则能够判断，例如按钮可见文本与 ARIA 标签语义不一致、图标按钮视觉含义不明确、页面布局导致阅读顺序混乱等问题，需要结合语义理解和视觉感知进行综合判断。

近年来，大语言模型和智能体技术的发展为复杂 Web 自动化任务提供了新的思路。大语言模型具备较强的自然语言理解、语义推理和结构化输出能力，能够对 HTML 片段、可访问性树和规则检测结果进行综合分析；多模态模型能够进一步结合页面截图和元素位置，发现视觉层面的可访问性问题；智能体框架则能够将任务规划、工具调用、状态记忆和结果反馈组织为可执行流程。将智能体技术引入 Web 无障碍检测，有望突破传统工具在动态遍历、语义理解和多步骤任务控制方面的限制。

基于上述背景，本文设计并实现 W4Agent 系统。系统面向 Web 无障碍自动化检测场景，采用多智能体协同架构，将检测任务拆分为规划、探索、检测和报告生成等阶段；通过 Playwright 浏览器自动化执行真实页面操作；通过启发式评分、页面结构去重和突发事件响应提高页面遍历效率；通过规则检测、大语言模型语义分析和多模态视觉分析构建多层次检测能力；通过前端工作台实现任务管理、实时监控、问题查看、截图标注和报告展示。该系统旨在探索一种“规则驱动”和“认知驱动”相结合的 Web 无障碍检测实现方案。

1.2 国内外研究现状

Web 无障碍检测研究和工具建设已经形成较为成熟的基础。当前常见自动化工具包括 axe-core、WAVE、Lighthouse、Pa11y 等。这类工具通常以 WCAG 标准为依据，通过分析页面 DOM、CSS、ARIA 属性、表单控件、链接文本、标题结构等信息，判断页面是否存在确定性无障碍缺陷。其优点在于检测速度快、规则结果可解释性强、容易集成到浏览器插件、命令行工具、持续集成流水线或自动化测试框架中。例如，axe-core 作为规则引擎可嵌入前端自动化测试流程；Lighthouse 能够在性能、SEO、最佳实践和无障碍等维度给出综合评分；WAVE 以可视化方式呈现页面中的错误、警告和结构信息；Pa11y 则适合命令行批量扫描。

然而，规则驱动工具也存在明显局限。首先，自动化工具一般只能覆盖部分 WCAG 成功准则。对于图片缺少 alt、表单缺少 label、页面没有 lang 属性等问题，规则可以较为准确地检测；但对于替代文本是否准确、链接文本是否能够脱离上下文表达目的、按钮标签是否符合视觉含义、页面阅读顺序是否符合用户理解路径等问题，规则难以直接判断。其次，现代 Web 应用大量使用前端路由、异步加载、组件库和复杂交互，检测工具如果只扫描初始页面，容易遗漏折叠菜单、弹窗表单、二级页面和用户操作后出现的内容。再次，传统工具对动态干扰元素的处理能力有限，Cookie 横幅、广告层、模态框或登录提示可能遮挡页面内容，甚至导致自动化流程停滞。最后，很多工具重在单页检测，对多页面任务编排、历史经验复用、实时监控和报告闭环支持不足。

在学术研究和工程实践中，针对上述问题出现了若干改进方向。第一类方向是增强爬虫和浏览器自动化能力，通过无头浏览器模拟用户行为，自动发现链接、按钮和表单入口，提高动态页面覆盖率。第二类方向是引入启发式策略，根据 URL、页面标题、控件数量、文本内容等信息判断页面价值，优先检测登录、注册、搜索、订单、表单等更可能出现无障碍问题的关键页面。第三类方向是将机器学习或语义分析用于问题识别，例如对替代文本质量、链接文本可理解性、图标语义等进行模型判断。第四类方向是加强可视化报告和人工复核，通过截图标注和问题状态管理降低测试人员定位问题的成本。

大语言模型和智能体技术的发展进一步推动了 Web 自动化测试方法的变化。LLM 能够读取自然语言任务描述、HTML 片段、浏览器状态和工具返回结果，并生成下一步操作计划或结构化分析结论。基于 LLM 的 Agent 不再只是固定执行脚本，而是可以在运行过程中根据页面变化和中间结果动态调整策略。在 Web 无障碍检测中，这种能力可以用于分析页面语义、解释规则结果、生成修复建议、判断下一步应继续探索还是开始检测，也可以结合长期记忆复用以往对同类网站或同一域名的检测经验。

尽管如此，LLM Agent 在工程落地中也面临挑战。一方面，模型调用存在成本和延迟，不适合替代所有规则检测；另一方面，模型输出可能存在不稳定性，需要通过规则约束、结构化输出、人工复核和置信度机制进行控制。因此，较合理的方案不是完全抛弃传统规则工具，而是将规则检测作为稳定底座，将 LLM 用于规则难以覆盖的语义和视觉问题，并通过智能体编排控制调用时机和上下文输入。本文所实现的 W4Agent 正是基于这一思路，在传统规则检测基础上叠加智能遍历、语义分析和视觉分析，以实现更完整的 Web 无障碍检测流程。

1.3 研究问题与挑战

图片占位：图 1-2 传统无障碍检测工具局限性示意图

建议内容：对比静态规则扫描与动态页面、弹窗、语义理解、视觉理解等挑战。

围绕 Web 无障碍自动化检测任务，本文需要解决的关键问题主要包括以下几个方面。

第一，复杂 Web 环境下的页面覆盖问题。传统检测流程往往以用户输入的起始 URL 为入口，对页面中的超链接进行广度优先或深度优先遍历。这类方法实现简单，但无法充分适应无障碍检测的目标差异。对于无障碍检测而言，表单页、登录页、注册页、搜索结果页、商品详情页、订单页等交互密集页面往往比普通文章页更有检测价值。如果遍历器按照固定顺序访问页面，可能在有限时间和深度内将大量资源消耗在低价值页面上，导致关键路径覆盖不足。因此，系统需要具备面向检测目标的自适应调度能力，能够根据 URL 特征、页面结构、交互元素和访问深度对候选页面进行排序。

第二，动态页面和突发事件处理问题。现代网站中常见 Cookie 同意横幅、订阅弹窗、广告浮层、登录提示、浏览器 Alert、异步加载和前端路由切换。这些元素可能阻挡真实内容，改变页面可访问性树，甚至使自动化浏览器无法继续执行后续动作。如果系统缺少突发事件响应机制，检测流程容易被中断，或者得到被遮挡状态下的不准确检测结果。因此，系统需要在页面导航和分析前后识别并处理常见干扰元素，尽可能恢复到用户可正常浏览的页面状态。

第三，重复页面和模板化页面控制问题。许多网站尤其是电商、资讯和列表类网站存在大量结构相似但内容不同的页面，例如商品详情页、文章详情页和用户资料页。如果系统对每个页面都完整执行截图、规则检测和 AI 分析，将导致检测耗时和模型调用成本快速上升。与此同时，完全按照 URL 判断重复也不可靠，因为不同 URL 可能对应相同模板，带有查询参数的 URL 也可能指向近似内容。因此，系统需要结合内容哈希和 DOM 结构签名，识别精确重复页面和结构模板重复页面，在保证覆盖代表性页面的同时避免无效重复检测。

第四，语义和视觉层面无障碍问题识别问题。规则检测擅长发现形式化问题，但很多可访问性缺陷具有语义性。例如，一个图片虽然包含 alt 属性，但 alt 内容可能是无意义文件名；一个按钮虽然包含 aria-label，但标签可能与图标真实含义不一致；一个链接文本可能仅写“点击这里”，脱离上下文后无法表达目标；页面视觉布局可能让用户误解阅读顺序。这类问题需要对文本、结构和视觉信息进行综合理解。系统需要在规则检测之后引入 AI 语义分析和多模态视觉分析，补充传统规则的不足。

第五，检测结果管理和人工复核问题。无障碍检测结果需要最终服务于开发修复和质量评估，仅给出问题列表并不能满足实际使用需求。测试人员需要了解问题出现在哪个页面、对应哪个元素、违反哪条规则、严重程度如何、如何修复，以及该问题是否为误报。对于视觉类问题，还需要通过截图标注快速定位。因此，系统需要提供从任务创建、实时监控、问题查看、截图标注、报告生成到人工复核的闭环能力。

第六，工程可扩展和可维护问题。无障碍标准和检测需求会持续变化，系统需要支持新增规则、替换模型、扩展 Agent、调整遍历策略和增加报告格式。如果各模块耦合严重，后续维护成本会显著增加。因此，系统在架构上需要采用清晰的分层设计，使前端展示、后端 API、Agent 编排、遍历器、检测引擎和数据持久化各自职责明确。

1.4 研究内容与主要贡献

针对上述问题，本文围绕 W4Agent 系统开展设计与实现工作，主要研究内容如下。

(1) 设计并实现基于多智能体协同的检测任务编排机制。系统将完整检测流程拆分为规划、探索、检测和报告生成等阶段，设置 Master Agent、Crawler Agent、Detector Agent 和 Reporter Agent 等角色。Master Agent 负责根据任务状态和中间结果动态决策下一步流程；Crawler Agent 负责调用遍历器完成页面探索；Detector Agent 负责结合规则结果和页面上下文进行语义分析；Reporter Agent 负责汇总问题并生成报告。该机制使检测流程不再是固定线性流水线，而是可以根据页面数量、检测进度和问题发现情况动态调整。

(2) 设计并实现面向无障碍检测的自适应智能遍历器。系统基于 Playwright 执行真实浏览器操作，能够获取 DOM、可访问性树、页面截图、链接和交互元素等信息。在遍历策略上，系统引入启发式 URL 优先级评分，对登录、注册、表单、搜索等高价值页面给予更高优先级；在去重策略上，系统结合内容哈希和 DOM 结构签名，减少模板化页面重复检测；在鲁棒性方面，系统引入突发事件响应器，处理 Cookie 横幅、弹窗、广告和 Alert 等干扰。

(3) 设计并实现规则-语义-视觉融合的无障碍检测引擎。系统首先通过规则注册机制执行确定性 WCAG 检测规则，发现图片替代文本、表单标签、标题层级、页面语言、地标结构、链接文本等常见问题；随后将规则结果、可访问性树和 HTML 片段输入 Detector Agent，由大语言模型进行语义层面分析；最后结合截图和元素边界框进行多模态视觉分析，补充视觉语义一致性和可见性相关问题。

(4) 设计并实现前后端分离的人机协同工作台。系统后端基于 FastAPI 提供项目、任务、页面、问题、报告和标注等 API，并通过 WebSocket 推送任务进度和 Agent 日志。前端基于 React、TypeScript 和 Ant Design 实现仪表盘、项目管理、任务创建、任务详情、报告查看、系统设置和截图标注组件，使测试人员能够直观地创建任务、跟踪执行过程、查看问题详情和复核检测结果。

(5) 设计系统测试和实验评估方案。本文从功能测试、模块测试和对比实验三个层面验证系统。功能测试关注任务创建、页面遍历、规则检测、报告生成和截图标注等核心流程；模块测试关注 Agent 编排、遍历器、检测引擎和前端交互；对比实验拟选择 axe-core、WAVE、Lighthouse 或 Pa11y 等工具，从页面覆盖率、问题发现数量、准确率、误报率和检测耗时等指标评估 W4Agent 的效果。

本文的主要贡献可以概括为：提出一种面向 Web 无障碍检测的多智能体协同流程；实现一种结合启发式评分、结构去重和事件响应的自适应遍历方法；构建规则检测、大语言模型语义分析和多模态视觉分析相结合的检测引擎；完成一套支持实时监控、报告查看和人工复核的工程化系统。

1.5 论文组织结构

本文共分为七章，各章内容安排如下。

第 1 章为绪论，介绍 Web 无障碍检测的研究背景和现实意义，分析国内外研究现状，总结传统检测工具在动态遍历、语义理解和结果闭环方面的不足，并给出本文的研究内容和主要贡献。

第 2 章为相关技术与理论基础，介绍 Web 无障碍标准、自动化检测方法、大语言模型与智能体技术、浏览器自动化与页面分析技术，以及系统开发所使用的主要技术栈。

第 3 章为系统需求分析，从系统目标、用户角色、功能需求、非功能需求和业务流程等角度，对 W4Agent 系统需要满足的需求进行分析。

第 4 章为系统总体设计，介绍系统整体架构、后端服务架构、智能体引擎、自适应遍历子系统、检测子系统、前端工作台、数据库和接口通信设计。

第 5 章为核心模块设计与实现，详细说明多智能体编排、记忆系统、自适应遍历器、页面分析、规则检测、AI 语义检测、多模态视觉分析、报告生成、前端工作台和部署配置等关键模块的实现过程。

第 6 章为系统测试与实验分析，介绍测试环境、功能测试、模块测试和对比实验设计，并从覆盖率、准确率、误报率、检测耗时等方面分析系统效果。

第 7 章为总结与展望，总结本文完成的主要工作和创新点，分析系统当前不足，并提出后续改进方向。

第 2 章 相关技术与理论基础

2.1 Web 无障碍标准

图片占位：图 2-1 WCAG 四大原则关系图

建议内容：以“可感知、可操作、可理解、健壮性”为中心，展示各原则与典型检测项。

Web 无障碍标准是系统设计和检测规则实现的依据。当前国际上应用最广泛的标准是 W3C Web Accessibility Initiative 发布的 Web Content Accessibility Guidelines，即 WCAG。WCAG 通过原则、指南、成功准则和充分技术等层次化结构，为 Web 内容的可访问性提供可验证要求。WCAG 的核心思想可以概括为四项原则：可感知、可操作、可理解和健壮性。

可感知原则要求信息和用户界面组件必须以用户能够感知的方式呈现。对于视觉障碍用户而言，图片、图标、图表等非文本内容需要提供文本替代；对于听障用户而言，音视频内容应提供字幕、转写或其他替代形式；对于低视力用户而言，文本和背景之间需要具备足够颜色对比度，页面缩放后仍应保持可读和可操作。可感知原则直接对应图片替代文本、颜色对比度、页面结构、媒体替代等检测项。

可操作原则要求用户界面组件和导航功能必须能够被操作。网站不能只依赖鼠标交互，也需要支持键盘访问；用户应能够跳过重复内容、理解当前焦点位置，并有足够时间完成操作；页面不应包含可能引发光敏性癫痫的闪烁内容。对于自动化检测系统而言，可操作性检测既包括静态规则，如交互元素是否可聚焦，也包括动态检查，如弹窗是否阻断键盘焦点、按钮是否能够通过键盘触发。

可理解原则要求信息和界面操作方式清晰可理解。页面需要声明自然语言，导航和交互模式应保持一致，表单输入错误应提供明确提示，链接文本和按钮名称应能够表达真实目的。许多可理解性问题具有明显语义属性，单纯通过 DOM 属性不一定能准确判断。例如，链接文本“更多”“点击这里”在某些上下文中可能可以理解，但脱离上下文后对屏幕阅读器用户并不友好。

健壮性原则要求内容能够被不同浏览器、辅助技术和用户代理可靠解析。页面应使用符合规范的 HTML 结构，合理使用 ARIA 属性，避免错误嵌套和无效角色。健壮性问题通常可以通过规则检测发现，例如元素角色与属性不匹配、ARIA 引用目标不存在、表单控件缺少可访问名称等。

WCAG 成功准则通常分为 A、AA、AAA 三个等级。A 级是基本无障碍要求，AA 级是多数组织和法规采用的目标等级，AAA 级则代表更高要求但并非所有内容都能完全满足。本文系统默认面向 AA 级检测，同时在数据模型和配置层面保留扩展到其他等级的能力。国内标准 GB/T 37668-2019《信息技术 互联网内容无障碍可访问性技术要求与测试方法》结合我国互联网应用实际，对网页内容、页面结构、导航操作、表单交互、兼容性和测

试方法提出要求。W4Agent 在检测项设计时综合考虑 WCAG 和国内标准，优先实现自动化程度较高、工程价值明确的规则，并通过 AI 分析补充语义类和视觉类问题。

2.2 自动化无障碍检测方法

图片占位：图 2-2 自动化无障碍检测方法分类图

建议内容：展示规则检测、动态检测、AI 语义分析、人工评估之间的关系。

自动化无障碍检测方法可以按照检测依据和执行方式分为规则检测、动态检测、智能分析和人工评估几类。规则检测是最常见的自动化方式，其核心是将无障碍标准转化为可执行规则，然后扫描页面 DOM、CSS、ARIA 属性、表单控件和链接结构。例如，检测 img 元素是否具有非空 alt 属性，检测 input 元素是否关联 label，检测 html 元素是否设置 lang 属性，检测标题层级是否跳跃，检测链接文本是否为空或过于模糊。规则检测的优点是确定性强、速度快、结果易解释，适合作为系统检测引擎的基础层。其不足在于只能发现形式化问题，无法判断替代文本质量、控件真实语义和视觉布局是否合理。

动态检测主要面向现代 Web 应用中的交互场景。许多页面内容并不会在初始 HTML 中出现，而是在用户点击菜单、展开折叠面板、切换标签页、提交表单或触发路由变化后动态渲染。如果检测系统只分析初始页面状态，就会遗漏大量实际用户会接触到的内容。动态检测通常需要借助浏览器自动化工具执行页面导航、点击、输入、滚动和截图等操作。相比静态规则扫描，动态检测能够获得更接近真实用户使用过程的页面状态，但也面临执行时间长、状态空间大、错误恢复复杂等问题。

智能分析是近年来随着机器学习和大语言模型发展而出现的重要方向。与规则检测不同，智能分析更关注语义质量和上下文理解。例如，模型可以判断图片替代文本是否与图像内容一致，按钮标签是否准确表达视觉功能，链接文本是否具备独立可理解性，页面是否存在阅读顺序混乱等问题。大语言模型还可以根据检测结果生成自然语言修复建议，帮助开发人员理解问题原因。然而，智能分析也存在模型输出不稳定、推理成本较高、误报难以完全避免等缺点。因此在工程系统中，智能分析更适合作为规则检测之后的补充层，而不是完全替代规则检测。

人工评估仍然是无障碍检测中不可缺少的环节。许多用户体验问题需要结合真实辅助技术、具体业务场景和人工判断才能确定。例如，一个替代文本是否“足够好”，取决于图片在页面中的功能；一个复杂表格是否易于屏幕阅读器理解，也需要结合实际朗读顺序进行判断。人工评估准确性较高，但成本高、效率低，不适合大规模频繁检测。W4Agent 的设计思路是以自动化检测降低初筛成本，通过截图标注、问题状态和人工标注机制支持测试人员对疑似问题进行复核，从而形成自动化与人工协同的工作流。

2.3 大语言模型与智能体技术

大语言模型是基于大规模语料训练得到的通用语言理解与生成模型，具备文本理解、逻辑推理、摘要生成、代码理解和结构化输出等能力。在 Web 无障碍检测场景中，大语言模型可以读取页面标题、HTML 片段、可访问性树、规则检测结果和上下文说明，分析页面是否存在语义层面的可访问性问题。例如，当规则引擎发现某页面存在多个链接文本均为“查看详情”时，模型可以结合上下文判断这些链接是否容易混淆；当表单控件具有标签但标签描述不清时，模型也可以给出更符合用户理解的修复建议。

智能体是在大语言模型基础上进一步封装任务感知、规划、工具调用、记忆和反馈机制的执行单元。普通模型调用通常是一次性问答，而智能体能够在任务执行过程中维护状态，根据中间结果决定下一步操作，并调用外部工具完成真实环境中的动作。在 W4Agent 中，智能体不是直接替代所有程序逻辑，而是与浏览器工具、检测工具、数据库和报告服务协同工作。Agent 负责更高层次的决策和语义分析，确定性执行仍由传统程序模块完成。

规划能力是智能体的重要特征。面对完整的网站检测任务，系统需要决定先探索哪些页面、何时开始检测、检测完成后是否继续探索、何时生成报告。如果采用固定流水线，系统难以根据任务实际状态调整流程。通过 Master Agent，系统可以根据已发现页面数、已测试页面数、已发现问题数、最大深度和任务状态等信息，动态选择 Explore、Test 或 Report 阶段，从而使检测流程更灵活。

工具调用使智能体能够与外部环境交互。在 Web 无障碍检测中，工具包括浏览器导航、页面截图、可访问性树提取、规则检测、数据库读写和报告生成等。智能体通过调用这些工具获取页面状态和执行结果，再将结果纳入下一步推理。为了保证可靠性，工具调用需要具有明确输入输出结构，并对异常情况进行处理。

记忆机制用于解决上下文连续性和经验复用问题。短期记忆保存当前任务中的页面状态、Agent 消息、检测结果和阶段信息，使不同 Agent 能共享任务上下文。长期记忆则保存跨任务经验，例如某域名历史上常见的问题类型、某类页面的处理策略、某些弹窗的关闭方式等。W4Agent 在设计上将记忆分为情景记忆、语义记忆和程序性记忆，分别对应具体检测经验、一般性知识和操作策略。这种设计使系统具备一定的持续学习和经验积累能力。

多智能体协作适合处理复杂任务。与单一 Agent 负责所有工作相比，多 Agent 架构可以将职责拆分得更清晰。Master Agent 负责全局调度，Crawler Agent 聚焦页面探索，Detector Agent 聚焦问题分析，Reporter Agent 聚焦结果汇总。不同 Agent 通过共享状态和记忆协同工作，既降低单个 Agent 的提示复杂度，也有利于系统扩展和维护。

2.4 浏览器自动化与页面分析技术

浏览器自动化技术是实现动态 Web 检测的重要基础。与传统 HTTP 爬虫不同，浏览器自动化工具能够加载 JavaScript、执行用户交互、处理前端路由、获取渲染后 DOM，并生成页面截图。Playwright 是现代浏览器自动化框架，支持 Chromium、Firefox 和 WebKit，提供页面导航、元素定位、点击输入、截图、网络等待和事件监听等能力。W4Agent 使用 Playwright 作为页面执行环境，以便在接近真实用户浏览的状态下完成页面分析。

DOM Tree 是浏览器解析 HTML 后形成的文档对象模型，表示页面元素的层级结构和属性。许多规则检测都基于 DOM Tree 完成，例如检查图片元素的 alt 属性、表单控件的 label 关联、标题元素的层级顺序、链接元素的文本内容等。DOM Tree 能够反映页面结构，但并不完全等同于辅助技术所感知的内容，因为某些元素可能被隐藏，某些语义可能来自 ARIA 属性，某些控件的可访问名称需要综合多个属性计算。

Accessibility Tree 是浏览器根据 DOM、ARIA、样式和可见性等信息生成的可访问性树，是屏幕阅读器等辅助技术理解页面的重要基础。相比 DOM Tree，Accessibility Tree 更接近辅助技术用户实际接收到的信息。它通常包含元素角色、可访问名称、状态、层级关系和可操作属性。对于无障碍检测而言，A11y Tree 可以帮助系统分析按钮、链接、表单、地标区域等是否具有合理语义，也是 LLM 进行语义分析的重要输入。

页面截图和元素边界框用于视觉层面检测和结果展示。许多问题需要结合视觉信息判断，例如元素是否被遮挡、文本对比度是否不足、图标按钮视觉含义是否与标签一致。通过 Playwright 获取页面截图，再结合元素 bounding box，系统可以将检测问题标注到截图上，生成直观的可视化结果。前端工作台展示标注截图后，测试人员可以快速定位问题元素，降低从问题描述回到页面代码的成本。

在动态页面检测中，浏览器自动化还需要处理等待和异常。页面加载完成并不意味着所有异步内容都已出现，检测器需要在导航、交互和截图之间设置合理等待策略；弹窗、Alert、跨域资源错误和网络超时也需要被捕获和处理。因此，W4Agent 将浏览器自动化封装为浏览器池、动作执行器和页面分析器等模块，使上层 Agent 和检测引擎能够以较稳定的方式使用浏览器能力。

2.5 系统开发技术栈

W4Agent 采用前后端分离和模块化设计，技术栈覆盖后端服务、数据持久化、智能体编排、浏览器自动化、前端展示和容器化部署等方面。

后端服务基于 Python 和 FastAPI 构建。FastAPI 支持异步请求处理、自动生成接口文档、基于类型注解的数据校验和较高性能的 Web 服务开发模式，适合构建检测任务管理、报告查询和 WebSocket 推送等接口。系统后端使用 SQLAlchemy AsyncIO 访问数据库，通过模型类描述项目、任务、页面、问题、报告、标注和 Agent 记忆等实体。数据库采用 PostgreSQL，并引入 pgvector 镜像以便后续支持向量化记忆和语义检索。Redis 用于缓存、状态管理或实时通知辅助能力。

智能体相关能力基于 LangChain 和 LangGraph 实现。LangChain 提供模型调用、提示词管理和工具封装能力，LangGraph 则适合将复杂 Agent 流程建模为状态图。W4Agent 将检测过程抽象为多个阶段和状态节点，由 Orchestrator 运行状态图，并由不同 Agent 完成探索、检测和报告生成等任务。系统的模型提供层支持不同 LLM Provider，便于根据成本、效果和部署条件替换模型。

浏览器自动化层使用 Playwright。Playwright 负责启动浏览器、打开页面、执行交互、提取页面结构、获取可访问性树和截图。为了降低频繁启动浏览器的开销，系统设计浏览器池复用浏览器实例。页面分析器和动作执行器在 Playwright 基础上进一步封装，使自适应遍历器能够以统一接口执行导航、分析、截图和事件处理。

前端采用 React 18、TypeScript、React Router、Ant Design、Ant Design Charts、Zustand、Axios 和 Vite。React 负责组件化界面构建，TypeScript 提升类型安全，React Router 管理页面路由，Ant Design 提供表格、表单、卡片、进度条、标签和布局等 UI 组件，Ant Design Charts 用于可视化统计，Axios 用于调用后端 API，Zustand 用于轻量级状态管理。前端页面包括仪表盘、项目列表、任务创建、任务详情、报告查看和系统设置等。

部署方面，系统使用 Docker 和 Docker Compose 组织服务。Docker Compose 文件中定义 PostgreSQL、Redis、Backend API 和 Frontend 服务。后端容器读取环境变量连接数据库和缓存，并将截图目录挂载到宿主机；前端容器构建静态资源并通过 Web 服务提供访问。容器化部署降低了环境配置复杂度，也便于后续在服务器或演示环境中快速启动完整系统。

第 3 章 系统需求分析

3.1 系统目标

W4Agent 的建设目标是实现一套面向 Web 应用的智能化无障碍检测系统，为测试人员和项目负责人提供从任务创建、自动遍历、问题检测、实时监控到报告生成的完整工作流。系统既要具备传统自动化检测工具的稳定性和规则可解释性，也要利用智能体和大语言模型提升动态页面适应能力、语义理解能力和复杂任务编排能力。

从业务目标看，系统需要帮助用户快速了解目标网站的无障碍质量状况。用户输入目标 URL、检测深度、检测等级和相关配置后，系统能够自动访问目标页面及其相关链接，发现页面中的可访问性问题，按照问题严重程度、规则类型、页面位置和修复建议进行组织，并生成可阅读的检测报告。对于大型网站或多页面应用，系统应尽可能避免只检测首页，而是覆盖具有代表性的关键页面和交互入口。

从技术目标看，系统需要实现多智能体协同检测流程。传统检测系统多采用“爬取—检测—报告”的线性架构，不利于根据中间状态调整策略。W4Agent 将检测任务拆分为多个阶段，由 Master Agent 统一调度 Crawler Agent、Detector Agent 和 Reporter Agent，使系统能够根据页面发现数量、检测进度和问题发现情况动态切换任务阶段。

从检测能力看，系统需要同时支持规则检测、语义检测和视觉分析。规则检测用于发现确定性、可解释的问题；语义检测用于分析替代文本质量、链接文本可理解性、表单说明清晰度等规则难以完全覆盖的问题；视觉分析用于识别截图中元素外观与可访问标签不一致、可见性不足或标注定位等问题。三类检测结果需要统一整合到问题模型中。

从交互目标看，系统需要提供易用的前端工作台。测试人员可以在工作台中创建项目和检测任务，查看任务运行状态、页面列表、问题列表、截图标注和报告内容；项目负责人可以通过仪表盘和报告了解整体质量；管理员可以在设置页面管理模型、浏览器和检测相关配置。

从工程目标看，系统需要具有良好的可扩展性和可维护性。规则库应支持新增规则，Agent 角色应支持扩展，模型提供方应支持替换，前后端模块应保持低耦合，部署方式应尽量标准化。基于这些目标，W4Agent 采用 FastAPI、SQLAlchemy、PostgreSQL、Redis、Playwright、LangGraph、React 和 Docker 等技术构建完整系统。

3.2 用户角色分析

根据系统使用场景，W4Agent 的主要用户角色可以分为系统管理员、测试人员和项目负责人三类。不同角色关注的功能重点不同，但都围绕无障碍检测任务展开。

系统管理员主要负责系统运行环境和全局配置管理。管理员需要配置数据库、缓存、浏览器池、截图目录、模型提供方、模型密钥、跨域访问和检测参数等内容。在系统运行过程中，管理员还需要关注服务状态、任务异常、模型调用失败和资源占用情况。对于后续扩展，管理员也可能负责维护检测规则、更新模型配置和调整部署环境。

测试人员是系统的主要使用者。测试人员负责创建项目，输入被测网站的基础 URL，配置检测任务名称、目标地址、最大遍历深度、检测等级和其他策略参数。任务运行后，测试人员需要通过任务详情页面观察实时进度、查看 Agent 执行日志、检查已发现页面和已检测页面数量。当任务完成后，测试人员需要查看问题列表、筛选严重程度、打开截图标注、确认问题是否真实存在，并对误报进行标注。测试人员还需要根据报告中的修复建议与开发人员沟通。

项目负责人主要关注检测结果和质量趋势。项目负责人不一定参与具体检测配置，但需要了解某个项目的整体无障碍合规情况，包括总页面数、总问题数、严重问题数量、问题类型分布、整改优先级和报告结论。对于多次检测，项目负责人还可能关注问题数量是否下降、严重问题是否修复以及不同版本之间的质量变化。

除上述主要角色外，系统也可以服务于前端开发人员和质量保障人员。前端开发人员可以根据问题详情和修复建议定位代码缺陷，质量保障人员可以将系统输出作为测试报告的一部分。由于毕业设计系统重点在检测流程和工程实现，权限体系可以采用简化设计，但在需求分析中仍需要明确不同角色对功能的关注差异，以便设计前端页面和后端接口。

3.3 功能需求分析

图片占位：图 3-1 W4Agent 系统用例图

建议内容：包含系统管理员、测试人员、项目负责人三个角色，以及项目管理、任务管理、检测分析、报告查看、人工标注等用例。

W4Agent 的功能需求围绕 Web 无障碍检测任务展开，可以划分为项目管理、任务管理、智能遍历、无障碍检测、实时监控、报告管理、人工标注和系统配置等模块。

项目管理模块用于组织被测对象。用户可以创建项目，填写项目名称、描述、基础 URL 和负责人等信息。一个项目可以包含多个检测任务，便于对同一网站在不同时间、不同配置或不同版本下进行多次检测。项目列表页

面需要展示项目基本信息、关联任务数量和最近检测情况，使用户能够快速进入具体项目或创建新任务。

任务管理模块是系统的核心入口。用户在创建任务时需要选择所属项目，填写任务名称和目标 URL，并配置最大遍历深度、检测等级、是否启用 AI 分析、是否启用视觉分析、最大页面数量等参数。任务创建后进入等待或运行状态，系统后端启动 Agent Engine 执行检测流程。任务状态包括 pending、running、paused、completed、failed 和 cancelled 等，系统需要记录任务进度、错误信息、已发现页面数、已检测页面数和已发现问题数。

智能遍历模块负责从目标 URL 出发自动探索页面。模块需要完成页面导航、页面结构分析、链接发现、候选 URL 排序、重复页面识别、弹窗处理和状态图维护。遍历过程中应优先访问具有较高检测价值的页面，例如包含 login、register、form、search、checkout 等特征的页面；对于重复模板页面，应在保留代表性样本的基础上减少重复检测；对于 Cookie 横幅、广告弹窗和 Alert，应尽可能自动关闭或跳过。

无障碍检测模块负责对已发现页面进行分析。系统首先执行规则检测，发现确定性问题；然后根据配置调用 Detector Agent 进行 AI 语义检测；若页面存在截图和元素位置信息，则可以进一步执行视觉分析。检测结果需要统一保存为问题记录，包括问题标题、描述、严重程度、WCAG 条款、问题类型、元素选择器、页面 URL、截图位置和修复建议等信息。

实时监控模块用于提升任务执行透明度。检测任务可能持续较长时间，用户需要了解当前任务是否正常运行。系统应通过 WebSocket 向前端推送任务状态变化、进度百分比、Agent 日志、页面发现事件和问题发现事件。前端任务详情页面应实时展示已发现页面、已测试页面、问题数量和当前阶段，避免用户只能等待最终结果。

报告管理模块用于汇总检测结果。任务完成后，系统需要生成报告，内容包括任务基本信息、检测范围、总页面数、总问题数、不同严重程度问题数量、不同 WCAG 条款问题分布、典型问题示例和修复建议。报告可以以页面形式展示，也可以后续扩展为 HTML、PDF 或 JSON 导出。

人工标注模块用于处理自动化检测的误报和复核需求。用户可以对某个问题进行确认、标记为误报、重新分类或添加备注。标注信息需要与问题关联保存，以便后续报告和模型优化使用。对于带有截图坐标的问题，前端应显示标注框和严重程度颜色，帮助用户快速定位。

系统配置模块用于管理运行参数。用户或管理员可以配置模型参数、检测等级、浏览器池大小、截图保存路径、API 地址等。虽然毕业设计实现可以保留部分配置在环境变量中，但从系统需求角度看，配置管理是保证系统适配不同环境的重要能力。

3.4 非功能需求分析

除功能需求外，W4Agent 还需要满足一系列非功能需求，以保证系统在实际使用中的稳定性和可维护性。

可扩展性方面，系统需要支持规则、Agent、模型和报告格式的扩展。无障碍标准会持续演进，不同项目也可能关注不同检测项，因此规则检测模块应采用注册机制，新增规则时不应修改大量核心代码。Agent 架构也应保持开放，例如后续可以增加专门的表单交互 Agent、登录 Agent 或成本控制 Agent。模型调用层需要隔离具体供应商，使系统可以根据效果和成本切换 OpenAI、Anthropic 或其他兼容模型。报告模块也应支持 HTML、PDF、JSON 等多种输出形式。

可靠性方面，系统需要能够处理复杂页面和异常情况。网页检测过程中可能出现网络超时、资源加载失败、页面跳转异常、浏览器崩溃、弹窗阻塞、LLM 调用失败或数据库写入失败。系统不应因为单个页面失败而终止整个任务，而应记录失败原因并继续处理其他页面。任务状态需要准确反映运行过程，当发生不可恢复错误时，应将任务标记为 failed 并保存错误信息。

易用性方面，系统前端需要降低用户理解成本。任务创建表单应清晰展示必要参数，任务详情页应通过进度条、统计卡片、日志列表和页面表格展示运行状态，报告页面应提供问题筛选和截图标注能力。对于无障碍问题，系统不应只给出技术字段，而应提供问题描述、影响说明和修复建议，帮助测试人员和开发人员理解。

性能方面，系统需要控制浏览器执行开销和模型调用成本。浏览器自动化比静态扫描消耗更多资源，因此系统需要通过浏览器池复用实例，通过页面去重减少重复检测，通过最大深度和最大页面数限制任务范围。AI 分析部分需要控制输入长度，例如只截取关键 HTML 片段，并在规则检测之后有选择地调用模型。对于实时监控，WebSocket 推送应避免过于频繁导致前端渲染压力。

可维护性方面，系统需要保持清晰分层和模块边界。前端负责展示和交互，后端 API 负责资源管理，服务层负责任务和报告逻辑，Agent 层负责智能编排，Crawler 层负责页面探索，Detector 层负责问题检测，Model 层负责数据持久化。清晰的目录结构和职责划分有助于后续调试、测试和功能扩展。

安全性方面，系统需要避免任意 URL 检测带来的潜在风险。在实际部署中，应限制访问内网敏感地址，避免 SSRF 类风险；模型密钥和数据库密码需要通过环境变量管理，不应硬编码在代码中；前端输入需要经过后端校验；截图和报告访问也应在生产环境中增加权限控制。毕业设计系统主要用于实验和演示，但在需求层面仍需要考虑这些安全要求。

3.5 业务流程分析

图片占位：图 3-2 检测任务业务流程图

建议内容：从创建项目、创建任务、Agent 编排、页面遍历、问题检测、报告生成到人工复核的完整流程。

W4Agent 的业务流程从用户创建检测任务开始，到系统生成检测报告结束，形成一个完整闭环。

首先，用户在前端工作台创建项目或选择已有项目，然后进入任务创建页面，输入目标 URL、任务名称、检测等级、最大遍历深度、是否启用 AI 检测等配置。前端通过 REST API 将任务创建请求发送到后端，后端校验参数后在数据库中创建任务记录，并将任务状态初始化为 pending。

其次，任务服务启动检测流程。后端根据任务配置创建 AgentEngine 实例，并将目标 URL 传入智能体编排器。任务状态更新为 running，同时通过 WebSocket 向前端推送任务开始事件。前端任务详情页面开始显示实时进度，包括当前状态、已发现页面数、已测试页面数和已发现问题数。

然后，Master Agent 根据初始状态制定流程决策。若当前尚未发现页面，则进入 Explore 阶段。Crawler Agent 调用自适应遍历器，浏览器池打开目标页面，事件响应器处理 Cookie 横幅和弹窗，页面分析器提取标题、URL、HTML、可访问性树、链接、交互元素和截图。去重器判断页面是否为重复页面，状态图记录页面节点和跳转关系，启发式策略对候选链接进行优先级排序。系统将有效页面保存到数据库，并更新任务的 pages_discovered 字段。

当系统发现可检测页面后，Master Agent 可以切换到 Test 阶段。Detector Agent 读取页面分析结果，首先调用规则检测引擎执行确定性 WCAG 规则，生成初步问题列表；随后根据配置调用大语言模型进行语义分析，将可访问性树、HTML 片段和规则问题作为上下文输入；如果启用视觉分析，则结合截图和元素边界框识别视觉层面的可访问性问题。检测结果经过合并和格式化后保存到 Issue 表中，同时更新页面状态和任务 issues_found 字段。

在任务运行过程中，Master Agent 会不断根据当前状态决定继续探索、继续检测或结束任务。如果仍存在高优先级未访问页面且未达到最大深度或最大页面数，系统继续 Explore；如果存在未测试页面，系统继续 Test；如果页面探索和检测均完成，系统进入 Report 阶段。

报告生成阶段由 Reporter Agent 或报告服务汇总任务结果。系统统计总页面数、总问题数、严重问题数量、主要问题数量、轻微问题数量、问题类型分布和页面结果，并生成摘要和修复建议。报告记录保存到数据库，必要时生成 HTML、PDF 或 JSON 文件路径。任务状态更新为 completed，并通过 WebSocket 通知前端。

最后，用户在报告页面查看检测结果。用户可以按照严重程度、规则类型和页面筛选问题，打开截图标注查看问题位置，对检测结果进行确认、误报标记、重新分类或添加备注。至此，一个检测任务完成闭环。该流程既体现了自动化检测能力，也保留了人工复核入口，符合实际无障碍测试 workflow。

第 4 章 系统总体设计

4.1 总体架构设计

图片占位：图 4-1 W4Agent 系统总体架构图

建议来源：优先使用 docs/W4Agent 系统架构图.png。

W4Agent 采用前后端分离和分层解耦的总体架构。根据系统职责划分，整体可以分为前端展示层、后端服务层、智能体引擎层、自适应遍历与检测层、数据与基础设施层。各层之间通过明确接口进行通信，既保证任务执行流程可以闭环，又降低了模块之间的耦合度。系统总体架构可结合项目中的 W4Agent 系统架构图进行说明。

前端展示层面向用户提供可视化工作台，主要负责项目管理、任务创建、任务实时监控、检测报告查看、问题筛选和截图标注等交互功能。前端不直接执行检测逻辑，而是通过 REST API 向后端提交任务和查询数据，通过 WebSocket 接收任务状态和 Agent 日志推送。这样的设计使前端专注于用户体验和数据展示，也便于后续替换为其他客户端或接入管理平台。

后端服务层基于 FastAPI 构建，是系统对外提供能力的统一入口。该层负责接收前端请求、校验输入参数、操作数据库、调度检测任务、管理报告和推送实时消息。后端服务层将不同资源拆分为项目、任务、页面、问题、报告、标注等 API 模块，并通过服务层封装任务执行、通知推送和报告生成等业务逻辑。后端服务层同时负责初始化 Redis、浏览器池和静态截图服务，为下层智能体执行提供运行环境。

智能体引擎层是 W4Agent 区别于传统检测工具的核心。该层以 AgentEngine 和 Orchestrator 为入口，基于 LangGraph 构建状态机，将检测流程组织为 route、explore、test 和 report 等节点。Master Agent 负责全局调度，Crawler Agent 负责页面探索，Detector Agent 负责无障碍问题分析，Reporter Agent 负责报告摘要和建议生成。多个 Agent 共享短期记忆和长期记忆，使任务上下文可以在不同阶段传递。

自适应遍历与检测层承担具体执行任务。自适应遍历器基于 Playwright 打开页面，执行导航、事件响应、页面分析、截图、链接发现、启发式排序和去重等操作。检测引擎先执行规则检测，再根据配置调用 AI 语义分析和视觉分析。该层既包含确定性程序逻辑，也包含模型推理能力，是系统完成“真实网页探索”和“问题发现”的关键。

数据与基础设施层包括 PostgreSQL、Redis、截图文件、报告文件和 Docker 容器运行环境。PostgreSQL 保存项目、任务、页面、问题、报告、标注和 Agent 记忆等结构化数据；Redis 用于缓存或实时消息辅助；截图文件和报告文件通过挂载目录保存；Docker Compose 负责组织数据库、缓存、后端和前端服务。该层保证系统数据可持久化、服务可部署、结果可追溯。

从数据流角度看，用户在前端提交检测任务后，后端创建任务记录并启动 AgentEngine。AgentEngine 调用 Orchestrator 进入状态图，先由路由节点判断下一步动作，再由 Crawler Agent 和 AdaptiveCrawler 发现页面，由 Detector Agent 和 DetectionEngine 生成问题，最后由 Reporter Agent 和 ReportService 汇总报告。任

务运行中的状态变化通过 NotificationService 推送到前端，检测结果则持久化到数据库，并在报告页面中展示。

4.2 后端服务架构设计

图片占位：图 4-2 后端服务分层架构图

建议内容：展示 FastAPI 路由层、Service 层、Agent Engine、数据库模型、Redis、PostgreSQL、截图静态资源之间的关系。

后端服务采用 FastAPI 作为 Web 框架，目录结构按照应用入口、路由层、服务层、数据模型、数据库连接和核心工具进行组织。`backend/app/main.py` 是应用入口，负责创建 FastAPI 实例、注册中间件、注册异常处理器、挂载 API 路由和 WebSocket 路由，并在生命周期函数中初始化 Redis 和浏览器池。应用启动时，系统会根据配置初始化 `BrowserPool`，使后续任务可以复用浏览器资源；应用关闭时，系统释放浏览器池、Redis 连接和数据库引擎，避免资源泄漏。

路由层位于 `backend/app/api/v1/`，按照资源类型划分为 projects、tasks、pages、issues、reports 和 annotations 等模块。统一路由文件将这些子路由注册到 `/api/v1` 前缀下，使前端能够以 REST 风格访问系统资源。项目接口负责项目的创建和查询；任务接口负责任务创建、状态查询和任务执行触发；页面接口负责已发现页面数据查询；问题接口负责检测问题查询和筛选；报告接口负责报告查看和生成；标注接口负责人工复核信息的保存。

WebSocket 路由位于 `backend/app/api/ws/`，主要用于任务实时监控。由于检测任务可能持续较长时间，如果前端只能轮询 REST API，会造成额外开销且实时性不足。因此系统通过 WebSocket 将任务状态、进度、Agent 日志和页面发现事件主动推送给前端。前端任务详情页建立连接后即可实时感知后端执行过程。

服务层位于 `backend/app/services/`，用于承载跨接口的业务逻辑。TaskService 负责任务生命周期管理和 AgentEngine 调度；NotificationService 负责向前端发送实时消息；ReportService 负责统计检测结果并生成报告；JiraService 则为后续对接外部缺陷管理系统保留扩展能力。通过服务层封装业务逻辑，可以避免路由函数直接包含复杂流程，使接口层保持简洁。

数据模型位于 `backend/app/models/`，基于 SQLAlchemy ORM 定义。模型覆盖 Project、Task、Page、Issue、Report、Annotation 和 AgentMemory 等实体，并通过外键和 relationship 描述实体关系。Schema 层位于 `backend/app/schemas/`，基于 Pydantic 定义请求和响应结构，保证接口输入输出具有清晰约束。数据库连接位于 `backend/app/db/`，采用异步 SQLAlchemy 会话与 PostgreSQL 通信。

后端还包含配置、日志、异常处理和安全等基础能力。`backend/app/config.py` 通过 Pydantic Settings 读取环境变量，统一管理数据库地址、Redis 地址、模型参数、浏览器池大小和截图目录等配置。

`backend/app/core/` 中的异常和日志模块用于提高系统可观测性。整体来看，后端服务层既是前端和智能体引擎之间的桥梁，也是系统数据一致性和任务生命周期管理的核心。

4.3 智能体引擎总体设计

图片占位：图 4-3 多智能体状态转移图

建议内容：展示 route、explore、test、report 节点，以及 Master/Crawler/Detector/Reporter Agent 的协作关系。

智能体引擎采用“统一入口 + 状态机编排 + 多 Agent 分工”的设计。`AgentEngine` 是任务服务调用智能体系统的主入口，它接收 `task_id` 和任务配置，内部创建 Orchestrator，并通过 `run(target_url)` 启动完整检测流程。这样的封装使后端任务服务无需了解 LangGraph 状态图细节，只需调用统一接口即可执行检测。

Orchestrator 是智能体编排的核心。系统定义 `PipelineState` 作为各节点共享状态，包含 `task_id`、`target_url`、`config`、`pages_discovered`、`pages_tested`、`issues_found`、`current_action`、`current_url`、`explored_urls`、`pending_urls`、`pending_test_urls`、`all_issues`、`report_data`、`error`、`iteration` 和 `max_iterations` 等字段。这些字段共同描述任务当前进度、待探索页面、待检测页面、已发现问题和错误信息。共享状态使不同节点之间可以通过结构化数据传递上下文，而不是依赖隐式全局变量。

状态图包括 `route`、`explore`、`test` 和 `report` 四类节点。`route` 节点负责更新迭代次数和当前动作，真正的路由决策由 `_deterministic_route` 函数完成。该函数按照优先级判断下一阶段：如果达到最大迭代次数则优先清空待检测页面并生成报告；如果待检测页面数量达到批量阈值则进入 `test`；如果仍有待探索 URL 且未达到最大页面数则进入 `explore`；如果没有待探索页面但仍有待检测页面则继续 `test`；如果所有页面都已处理则进入 `report`。该路由采用确定性规则而非 LLM 决策，能够避免模型不稳定导致流程失控，同时保留 Agent 在具体分析任务中的语义能力。

`explore` 节点负责页面探索。该节点会批量取出 `pending_urls` 中的 URL，调用 `AdaptiveCrawler` 探索页面，获取页面标题、链接、截图、可访问性树和其他页面数据。探索成功的 URL 会加入 `explored_urls` 和 `pending_test_urls`，新发现的 URL 会加入 `pending_urls`。节点执行后返回 `route` 节点，由路由函数重新判断下一步动作。

`test` 节点负责页面检测。该节点会批量取出 `pending_test_urls` 中的 URL，读取或获取页面数据，调用 `DetectionEngine` 执行规则检测和 AI 检测，并将问题追加到 `all_issues` 中。同时系统会更新 `pages_tested` 和 `issues_found` 等进度字段。`test` 节点执行完毕后同样回到 `route` 节点，使系统可以在探索和检测之间交替运行。

`report` 节点负责报告生成。系统汇总已检测页面和问题列表，计算总页面数、总问题数、严重程度分布和综合得分，再调用 `Reporter Agent` 生成自然语言摘要和修复建议。报告数据写入状态中的 `report_data`，并由后续报告服务持久化。

Agent 角色方面，`Master Agent` 代表全局调度者，负责理解任务状态和输出高层决策；`Crawler Agent` 关注页面探索策略；`Detector Agent` 关注无障碍问题分析；`Reporter Agent` 关注报告摘要和建议。四类 Agent 共享 `ShortTermMemory` 和 `LongTermMemory`。短期记忆保存当前任务中的关键事件，长期记忆保存跨任务经验，使系统具备上下文感知和经验复用能力。

4.4 自适应遍历子系统设计

图片占位：图 4-4 自适应遍历子系统结构图

建议内容：展示 `AdaptiveCrawler`、`BrowserPool`、`ActionExecutor`、`PageAnalyzer`、`HeuristicStrategy`、`Deduplicator`、`EventResponder`、`StateGraph`。

自适应遍历子系统的目标是在有限检测资源下尽可能覆盖有价值的页面，并保证遍历过程能够适应动态 Web 环境。该子系统主要由 `AdaptiveCrawler`、`BrowserPool`、`ActionExecutor`、`PageAnalyzer`、`HeuristicStrategy`、`SemanticDeduplicator`、`EventResponder` 和 `ApplicationStateGraph` 组成。

`AdaptiveCrawler` 是遍历流程的协调者。它在初始化时读取 `max_depth` 和 `max_pages` 等配置，创建状态图、启发式策略、语义去重器、事件响应器和页面分析器。探索单个页面时，`AdaptiveCrawler` 从 `BrowserPool` 获取页面实例，由 `ActionExecutor` 执行导航；导航成功后先调用 `EventResponder` 处理弹窗和横幅，再调用 `PageAnalyzer` 分析页面结构；随后使用 `SemanticDeduplicator` 判断是否重复；如果页面有效，则更新状态图、截图、发现链接、排序候选 URL、保存页面数据并推送进度通知。

`BrowserPool` 用于复用浏览器资源。浏览器启动和关闭成本较高，如果每访问一个页面都新建浏览器，会显著降低检测效率。浏览器池在应用启动时初始化多个页面或上下文，任务执行时通过 `get_page` 获取可用页面，执

行完成后 `release_page` 释放回池中。这样既减少资源开销，也便于集中管理浏览器生命周期。

`ActionExecutor` 封装页面动作，包括导航、获取链接、点击和截图等操作。它屏蔽 Playwright 的底层调用细节，使 `AdaptiveCrawler` 能够通过稳定接口执行页面访问。`PageAnalyzer` 负责提取页面数据，包括 URL、标题、HTML、链接、表单、图片、标题结构、地标区域、可访问性树和元素边界框等。这些数据既用于规则检测，也用于 AI 语义分析和视觉分析。

`HeuristicStrategy` 负责候选 URL 的优先级排序。无障碍检测并不要求平均访问所有页面，而是应优先访问交互密集和风险较高的页面。因此，策略会根据 URL 关键词、深度和页面价值进行评分。例如，包含 `login`、`register`、`form`、`search`、`checkout` 等特征的页面通常与用户输入、身份认证或关键业务流程相关，更容易出现表单标签、键盘操作和错误提示等问题，应获得更高优先级；深度过深的页面则适当降低优先级，以防止遍历无限扩张。

`SemanticDeduplicator` 用于减少重复检测。系统先使用内容哈希识别完全重复页面，再使用 DOM 结构签名识别模板化页面。对于电商商品页、新闻详情页等结构高度相似的页面，系统可以保留少量代表性样本，而不必对每个 URL 都执行完整检测。该机制能够降低浏览器执行时间、数据库写入量和 LLM 调用成本。

`EventResponder` 用于处理动态干扰元素。页面加载后，系统会检查 Cookie 横幅、模态弹窗、广告层和 Alert 等常见阻塞元素，并尝试点击同意、关闭或取消按钮。如果突发事件不能被处理，系统也会记录日志并尽量继续后续分析。事件响应机制提高了遍历器在真实网站环境中的鲁棒性。

`ApplicationStateGraph` 维护页面访问图。每个页面作为节点，页面之间的链接或操作作为边，节点状态包括待访问、已探索、失败、跳过等。状态图不仅帮助系统避免重复访问，也为后续报告中的页面覆盖分析和访问路径追踪提供基础。

4.5 检测子系统设计

图片占位：图 4-5 规则-语义-视觉三层检测架构图

建议内容：展示规则检测、LLM 语义检测、视觉分析三层输入、处理和输出。

检测子系统采用规则检测、AI 语义检测和多模态视觉分析相结合的三层架构。该设计的出发点是充分利用规则检测的稳定性，同时引入大语言模型弥补语义和视觉理解不足。

第一层是规则检测。`DetectionEngine` 的 `detect` 方法首先导入 WCAG 规则模块以触发规则注册，然后调用 `Rule Registry` 对页面数据执行所有符合等级要求的规则。规则检测输入来自 `PageAnalyzer`，包括图片列表、表单控件、标题结构、链接列表、页面语言、地标区域等。当前规则覆盖图片替代文本、表单标签、标题层级、页面语言、地标结构和链接文本等常见问题。规则检测结果包含 `rule_id`、`wcag_criterion`、`wcag_level`、`severity`、`title`、`description`、`recommendation` 和 `detected_by` 等字段，具有较强可解释性。

第二层是 AI 语义检测。`DetectionEngine` 的 `detect_full` 方法在规则检测之后创建 `DetectorAgent`，并将 URL、标题、可访问性树、HTML 片段和规则问题作为输入。`DetectorAgent` 基于提示词要求大语言模型从 WCAG 原则出发分析规则可能遗漏的问题，例如替代文本是否准确、链接文本是否清晰、表单说明是否充分、页面结构是否容易理解等。AI 检测结果与规则检测结果合并，形成更完整的问题列表。

第三层是多模态视觉分析。视觉分析模块读取页面截图和元素边界框信息，将元素文本、`alt`、`aria-label`、`role` 和 `bbox` 等摘要与截图一起输入视觉模型。模型分析元素外观和可访问名称之间是否一致，判断按钮、图标、链接、颜色对比和布局是否存在视觉层面的可访问性风险。视觉分析特别适合发现纯 DOM 信息难以判断的问题，例如图标按钮缺少可理解标签、视觉按钮不可聚焦、文本与背景对比不足、重要元素被遮挡等。

三层检测结果最终统一到 Issue 数据模型中。系统需要对不同来源的问题进行合并和去重，避免同一问题被规则和 AI 重复报告。对于确定性问题，优先保留规则检测结果；对于语义和视觉问题，保留模型给出的解释和建议，并在前端标注其来源。通过 detected_by 字段区分 rule、ai 和 vision，有助于测试人员理解问题可信度并进行复核。

这种分层架构具有良好的工程优势。规则层提供稳定底座，保证常见问题可以快速检测；语义层扩展检测深度，提升对可理解性问题的发现能力；视觉层补充页面截图视角，提升问题定位和展示效果。三层之间并非简单替代关系，而是互相补充，共同服务于更全面的无障碍评估。

4.6 前端工作台设计

图片占位：图 4-6 前端页面路由与工作台结构图
建议内容：展示 Dashboard、ProjectList、TaskCreate、TaskDetail、ReportView、Settings、AnnotatedScreenshot 的页面关系。

前端工作台采用 React、TypeScript、React Router 和 Ant Design 构建。frontend/src/App.tsx 定义了系统主要路由，包括首页仪表盘、项目管理、任务创建、任务详情、报告查看和系统设置。所有页面都包裹在 AppLayout 中，保证导航栏、侧边栏和内容区域风格统一。

Dashboard 页面作为系统首页，展示全局统计信息，例如项目数量、任务数量、已发现问题数量、严重问题数量和近期检测趋势。通过卡片和图表，用户可以快速了解系统整体运行情况和无障碍质量概况。

ProjectList 页面用于项目管理。用户可以查看已有项目、项目基础 URL、负责人和创建时间，也可以创建新项目或进入项目相关任务。项目作为任务的组织单位，适合对同一网站进行多轮检测和历史对比。

TaskCreate 页面用于创建检测任务。用户选择项目后填写目标 URL 和任务名称，并配置检测深度、页面数量、检测等级、是否启用 AI 分析等参数。表单提交后，前端调用后端任务 API 创建任务，并跳转到任务详情页面查看执行状态。

TaskDetail 页面是实时监控的核心页面。它通过 REST API 获取任务基础信息，通过 WebSocket 接收后端推送的进度和日志。页面展示任务状态、进度条、已发现页面数、已检测页面数、问题数量、Agent 日志和页面列表。用户可以在任务运行过程中了解系统当前处于探索、检测还是报告生成阶段。

ReportView 页面用于展示检测报告。报告内容包括总体得分、页面数量、问题数量、严重程度分布、问题类型分布、问题列表和修复建议。用户可以筛选问题、查看问题详情、打开相关页面截图，并根据问题描述进行修复。

Settings 页面用于展示或配置系统参数，例如 API 地址、模型配置、检测默认值等。虽然部分配置在当前实现中由环境变量管理，但设置页为后续产品化提供了入口。

AnnotatedScreenshot 组件用于展示标注截图或在截图上叠加问题位置。对于带有 bounding box 的问题，组件可以通过矩形框和颜色区分严重程度，使测试人员直观看到问题所在区域。与纯文本问题列表相比，截图标注显著提高了问题定位效率。

4.7 数据库设计

图片占位：图 4-7 数据库 ER 图
建议内容：根据 backend/app/models/ 绘制 Project、Task、Page、Issue、Report、Annotation、AgentMemory 之间的关系。

数据库设计围绕检测任务闭环展开，核心实体包括 Project、Task、Page、Issue、Report、Annotation 和 AgentMemory。各实体通过外键建立关联，使系统能够从项目追踪到任务，从任务追踪到页面和问题，再从问题追踪到人工标注。

Project 表保存被测项目基本信息，包括项目名称、描述、基础 URL 和负责人。一个项目可以包含多个 Task，因此 Project 与 Task 是一对多关系。该设计便于同一网站进行多次检测，也便于后续统计项目维度的质量变化。

Task 表保存检测任务信息，包括所属项目、任务名称、目标 URL、任务状态、配置、已发现页面数、已检测页面数、已发现问题数和错误信息。任务状态枚举包括 pending、running、paused、completed、failed 和 cancelled，能够描述任务生命周期。config 字段使用 JSON 存储最大深度、最大页面数、检测等级、是否启用 AI 等灵活配置。

Page 表保存被遍历页面信息，包括任务 ID、URL、标题、状态、深度、内容哈希、语义相似度、截图路径、标注截图路径、可访问性树和页面元数据。页面状态包括 discovered、crawling、testing、completed、failed 和 skipped，用于反映页面在遍历与检测流程中的位置。Page 与 Issue 是一对多关系，一个页面可以产生多个问题。

Issue 表保存无障碍问题信息。问题记录通常包括任务 ID、页面 ID、规则编号、WCAG 条款、严重程度、标题、描述、建议、元素选择器、截图定位、检测来源和状态等内容。通过 task_id 和 page_id，系统既可以按任务统计问题，也可以按页面展示问题。

Report 表保存任务报告，包括整体得分、A/AA/AAA 等级得分、总页面数、总问题数、严重问题数、主要问题数、轻微问题数、摘要、建议、报告文件路径、问题分布和页面结果等。Report 与 Task 是一对一关系，表示每个检测任务最终生成一份报告。

Annotation 表保存人工复核信息。标注类型包括 confirm、false_positive、reclassify 和 comment，分别表示确认问题、标记误报、重新分类和添加备注。人工标注与 Issue 关联，使系统能够区分自动检测结果和人工确认结果。

AgentMemory 表保存长期记忆，字段包括 memory_type、domain、key、content、metadata_json 和 relevance_score。记忆类型包括 episodic、semantic 和 procedural，分别表示具体检测经历、一般性模式知识和处理策略。该表为 Agent 跨任务经验复用提供数据基础。

4.8 接口与通信设计

图片占位：图 4-8 REST API、WebSocket 与静态资源通信示意图

建议内容：展示前端、后端 API、WebSocket、截图静态服务和数据库之间的数据流。

系统通信设计包括 REST API、WebSocket 和静态资源访问三部分。三者分别服务于资源操作、实时消息推送和截图报告访问。

REST API 是前后端交互的主要方式。后端统一以 `/api/v1` 为前缀组织接口，按资源划分为 projects、tasks、pages、issues、reports 和 annotations 等模块。前端通过 Axios 调用这些接口，完成项目创建、任务创建、任务查询、页面查询、问题筛选、报告查看和标注提交等操作。REST API 适合处理请求-响应式业务，具有结构清晰、易调试和易扩展的特点。

WebSocket 用于任务实时监控。检测任务执行时间较长，并且运行过程中会不断产生状态变化和日志。如果前端频繁轮询任务接口，不仅实时性有限，也会增加后端压力。因此系统为任务监控提供 WebSocket 通道。任务

开始、页面发现、检测完成、问题发现、Agent 日志和任务结束等事件由 NotificationService 推送给前端。前端收到消息后更新进度条、日志列表和统计卡片。

静态资源访问用于截图和标注图片展示。后端在 FastAPI 应用中将截图目录挂载为静态文件路径，例如 `/static/screenshots`。页面分析和视觉标注生成的图片保存到服务器文件系统，数据库中保存相对路径或文件名，前端在报告和问题详情中通过静态 URL 加载图片。该设计避免将大图片直接存入数据库，同时便于前端展示和后续文件清理。

接口与通信设计需要考虑一致性和异常处理。任务状态以数据库为准，WebSocket 消息用于实时更新；如果前端断开连接，重新进入任务详情页时仍可以通过 REST API 获取最新状态。截图文件路径需要与数据库记录保持一致，若文件不存在，前端应显示降级提示。通过 REST API、WebSocket 和静态资源的组合，系统同时满足了管理操作、实时体验和可视化展示需求。

第 5 章 核心模块设计与实现

5.1 多智能体协同编排模块实现

图片占位：图 5-1 Agent 编排执行流程图

建议内容：展示 AgentEngine 启动、Orchestrator 状态图、Explore/Test/Report 批处理与回流路由。

多智能体协同编排模块是 W4Agent 的流程控制核心。系统将一次完整检测任务抽象为状态驱动的多阶段过程，并通过 AgentEngine 对外提供统一执行入口。后端任务服务在启动检测时创建 AgentEngine，传入 `task_id` 和任务配置，然后调用 `run(target_url)`。AgentEngine 内部创建 Orchestrator，由 Orchestrator 负责构建和运行 LangGraph 状态图。

Orchestrator 初始化时会创建共享 LLM、短期记忆、长期记忆和 AdaptiveCrawler 实例。随后分别实例化 MasterAgent、CrawlerAgent、DetectorAgent 和 ReporterAgent。所有 Agent 都接收相同的 `task_id` 和共享记忆对象，因此它们可以在同一任务上下文中协作。共享 AdaptiveCrawler 的设计也有利于复用页面缓存和状态图，避免探索和检测阶段重复初始化遍历器。

状态图的节点包括 `route`、`explore`、`test` 和 `report`。`route` 节点本身不执行复杂 LLM 推理，而是更新迭代状态并交给确定性路由函数判断下一步。这样做的原因是流程控制属于高可靠要求模块，如果完全依赖 LLM 输出，可能出现阶段跳转不稳定、重复执行或提前结束等问题。系统采用规则化路由，将 LLM 的能力保留在语义分析和报告生成等更适合模型发挥的环节。

确定性路由函数的规则体现了任务执行优先级。首先，如果迭代次数达到上限，系统会停止继续探索，优先处理剩余待检测页面，然后进入报告阶段，以防止异常链接导致无限循环。其次，如果待检测页面数量达到批量阈值，说明已积累足够页面，应优先检测，避免只探索不分析。再次，如果仍存在待探索 URL 且未超过最大页面数，则继续探索。最后，当探索和检测均完成后，系统进入报告阶段。

`explore` 节点以批量方式处理待探索 URL。每次执行最多探索若干页面，探索成功后将页面加入 `explored_urls`，并把该页面加入 `pending_test_urls`，表示后续需要检测。探索过程中发现的新 URL 会被加入 `pending_urls`，由后续 `route` 决策是否继续访问。批处理设计能够在探索效率和状态反馈之间取得平衡，既避免每个页面都频繁切换状态图，也避免一次探索过多页面导致用户长时间无法获得进度更新。

`test` 节点以批量方式处理待检测页面。节点读取页面数据，调用 DetectionEngine 执行规则检测和 AI 检测，并将问题写入 `all_issues`。每检测完成一个页面，系统更新 `pages_tested` 和 `issues_found`，并通过通知服务向前端推送进度。检测失败时，系统记录错误但尽量不中断整个任务，保证单页失败不会全局结果。

report 节点汇总任务结果。系统统计页面数、问题数和严重程度分布，计算综合分数，并调用 ReporterAgent 生成摘要和修复建议。ReporterAgent 的提示词要求模型根据检测统计生成面向人类读者的报告内容。如果模型输出 JSON 解析失败，系统提供降级摘要和默认建议，保证报告阶段具备容错能力。

从实现效果看，多智能体协同编排使 W4Agent 具备了动态任务控制能力。相比固定流水线，该模块可以根据待探索页面、待检测页面和迭代次数调整阶段顺序；相比完全由 LLM 决策，该模块又通过确定性状态机保证流程稳定。两者结合符合工程系统对可靠性和智能性的双重要求。

5.2 记忆系统实现

记忆系统用于支撑 Agent 在任务执行过程中的上下文共享和跨任务经验复用。W4Agent 将记忆分为短期记忆和长期记忆两类。短期记忆面向单次任务生命周期，长期记忆面向多个任务之间的经验积累。

短期记忆通常以有界队列或内存结构保存最近发生的事件和 Agent 输出。一次检测任务可能包含多个阶段：页面探索、规则检测、语义分析和报告生成。如果不同 Agent 之间完全没有共享上下文，Detector Agent 就无法理解 Crawler Agent 发现页面的背景，Reporter Agent 也无法利用前面阶段的总结。短期记忆保存如“页面已探索”“发现某类问题”“某个弹窗已处理”“某页面检测失败”等事件，使后续 Agent 能够在上下文中做出更合理判断。

短期记忆的容量需要受到控制。由于 LLM 输入上下文有限，不能无限累积所有日志和页面内容。系统采用有界结构保存关键事件，并在需要时选取与当前任务最相关的信息。这样既避免内存增长，也减少模型调用输入长度。

长期记忆通过 AgentMemory 数据表持久化。系统将记忆类型划分为 episodic、semantic 和 procedural。情景记忆记录具体检测经历，例如某个域名的某次检测中发现大量图片替代文本问题；语义记忆记录从多个任务中总结出的通用模式，例如电商网站商品详情页常见图片 alt 缺失；程序性记忆记录处理策略，例如某类 Cookie 横幅应点击“接受”按钮关闭。

长期记忆的字段包括 domain、key、content、metadata_json 和 relevance_score。domain 用于按网站或业务域划分记忆，key 用于检索具体记忆，content 保存自然语言或结构化经验，metadata_json 保存附加信息，relevance_score 表示记忆重要程度。后续任务检测同一域名或相似页面时，Agent 可以检索相关记忆，辅助决策和分析。

在当前系统中，记忆系统主要服务于 Agent 上下文共享和报告生成。未来可以进一步结合向量检索，将页面结构、问题类型和处理策略转化为向量表示，通过语义相似度检索历史经验。例如，当系统遇到一个结构类似的登录页时，可以检索历史上登录页常见问题和弹窗处理方式，从而提高检测效率和结果一致性。

5.3 自适应智能遍历器实现

图片占位：图 5-2 自适应遍历器单页探索流程图

建议内容：导航、事件响应、页面分析、去重、截图、链接发现、评分排序、保存数据库、通知前端。

自适应智能遍历器的实现目标是在真实浏览器环境中自动发现页面，并根据检测价值优先访问关键页面。遍历器以 AdaptiveCrawler 为核心，围绕单页探索流程展开。

探索开始时，AdaptiveCrawler 从浏览器池获取一个页面实例，并通过 ActionExecutor 导航到目标 URL。导航结果包含是否成功、最终 URL 和错误信息。若导航失败，系统将状态图中对应节点标记为失败，并返回失败结果；若导航成功，则进入事件响应和页面分析阶段。

页面加载后，EventResponder 会检查常见突发事件。许多网站会在首次访问时显示 Cookie 横幅、营销弹窗或广告遮罩，这些元素可能遮挡正文和控件。如果直接分析页面，检测结果可能反映的是弹窗而非真实页面。事件响应器会尝试识别关闭、接受、同意、取消等按钮并执行点击，对于浏览器 Alert 也会尝试处理。该步骤提高了后续页面分析的准确性。

随后 PageAnalyzer 提取页面数据，包括标题、URL、HTML、链接、图片、表单、标题结构、地标区域、可访问性树和元素信息。页面数据是后续去重、检测和报告的基础。系统还会为有效页面保存截图，以便报告中展示和视觉分析使用。

去重阶段由 SemanticDeduplicator 完成。系统首先根据页面内容计算哈希，用于识别完全重复页面；然后根据 DOM 结构生成模板签名，用于识别结构相似的页面。对于被判定为重复的页面，系统会在状态图中标记为 skipped，并缓存必要页面数据，但不再继续发现子链接和执行完整检测。这样可以避免大量模板页消耗检测资源。

对于非重复页面，AdaptiveCrawler 将其加入 ApplicationStateGraph，并标记为已探索。状态图节点保存 URL、标题、深度、父节点和状态等信息，边表示从一个页面到另一个页面的链接关系。状态图的作用不仅是避免重复访问，还能记录页面探索路径，为报告中的覆盖情况分析提供依据。

链接发现由 ActionExecutor 获取页面中所有同域或允许范围内的链接。系统根据当前页面深度判断是否继续扩展。如果未达到最大深度，则调用 HeuristicStrategy 对候选 URL 评分和排序。评分策略通常以基础分为起点，根据 URL 关键词、页面深度和业务价值进行加减分。包含 login、signup、register、form、contact、search、cart、checkout 等关键词的 URL 通常具有较高交互价值，优先级更高；深度越大，优先级越低，以控制遍历范围。

最后，系统将高优先级且未访问的新 URL 加入状态图和待探索队列，并保存当前页面到数据库。保存内容包括 URL、标题、深度、截图路径、可访问性树和元数据等。系统同时通过 NotificationService 通知前端发现了新页面。至此，单页探索流程结束，页面实例释放回浏览器池。

5.4 页面分析与动作执行实现

页面分析与动作执行模块负责将 Playwright 的底层能力封装为系统可复用接口。该模块主要包括 BrowserPool、ActionExecutor 和 PageAnalyzer。

BrowserPool 用于管理浏览器资源。浏览器自动化检测需要打开真实浏览器页面，如果每次探索都启动新浏览器，会导致大量时间和内存开销。系统在应用生命周期启动阶段初始化浏览器池，根据配置创建一定数量的浏览器上下文或页面。任务执行时，AdaptiveCrawler 通过 `get_page` 获取可用页面，执行完毕后通过 `release_page` 归还。应用关闭时统一调用 shutdown 释放资源。

ActionExecutor 封装浏览器动作。导航动作负责访问目标 URL，并处理超时、跳转和错误返回；链接提取动作负责从当前页面获取所有候选链接，并根据 base_url 过滤外域链接或无效链接；截图动作负责保存页面当前视觉状态；后续还可以扩展点击、输入、滚动等动作，以支持更复杂的交互探索。ActionExecutor 将 Playwright 调用结果包装为结构化字典，使上层遍历器能够统一处理成功和失败情况。

PageAnalyzer 是页面信息提取的核心。它在页面加载并处理弹窗后运行，提取用于无障碍检测的多种数据。DOM 层面，分析器获取 HTML 片段、图片元素、链接元素、表单控件、标题结构和地标元素；辅助技术层面，分析器获取 Accessibility Tree；视觉层面，分析器获取可见元素的边界框、文本、角色、alt 和 aria-label 等信息；页面元数据层面，分析器获取 URL、标题、语言和其他 meta 信息。

截图保存是连接检测和展示的重要环节。AdaptiveCrawler 在页面确认非重复后调用截图逻辑，生成唯一文件名并保存到配置指定的截图目录。数据库 Page 记录保存 screenshot_path，前端报告页面通过静态资源接口加载

截图。对于视觉检测或问题定位，系统可以在原始截图基础上绘制标注框，生成 `annotated_screenshot_path`。

元素提取需要兼顾规则检测和视觉分析。规则检测关注元素属性，例如 `img` 的 `alt`、`input` 的 `label`、`a` 的文本、`h1-h6` 的层级等；视觉分析关注元素位置和外观，因此需要 `bounding box`、文本截断、角色和可访问名称等信息。通过在 `PageAnalyzer` 中统一提取这些数据，系统可以避免不同模块重复访问页面，提高效率。

该模块的设计体现了底层工具封装思想。`Playwright` 功能强大但接口较底层，直接在 `Agent` 或检测引擎中调用会导致耦合过高。将浏览器池、动作执行和页面分析拆分后，上层模块只需关注“探索页面”和“获取页面数据”，不必关心浏览器细节，从而提高系统可维护性。

5.5 规则检测引擎实现

图片占位：图 5-3 规则注册与执行流程图

建议内容：展示 `A11yRule`、`Rule Registry`、`WCAG Rules`、`DetectionEngine` 和 `Issue` 输出结构。

规则检测引擎是 `W4Agent` 的基础检测层。其核心由 `Rule Registry`、`A11yRule` 基类和具体 `WCAG` 规则组成。规则引擎的职责是对 `PageAnalyzer` 输出的结构化页面数据执行确定性检查，并返回统一格式的问题列表。

`Rule Registry` 采用注册表模式管理规则。每条规则作为一个类实现统一接口，并在模块导入时注册到全局 `registry` 中。`DetectionEngine` 执行检测时导入 `detector.rule_based.wcag_rules`，触发规则注册，然后调用 `registry.run_all(page_data, wcag_level)` 批量执行规则。注册表模式使新增规则更加方便，只需定义新的规则类并注册即可，不需要修改 `DetectionEngine` 主流程。

`A11yRule` 基类定义规则的基本属性和检查接口。每条规则通常包含 `rule_id`、`wcag_criterion`、`wcag_level` 和 `description` 等元数据。`rule_id` 用于系统内部识别，`wcag_criterion` 对应 `WCAG` 成功准则，`wcag_level` 表示规则等级，`description` 描述规则含义。具体规则通过实现 `check(page_data)` 方法返回问题列表。

当前 `WCAG` 规则覆盖多个常见检测项。图片替代文本规则检查图片是否具有有意义的 `alt` 属性；表单标签规则检查 `input`、`select`、`textarea` 等控件是否关联 `label`、`aria-label` 或 `title`；标题结构规则检查页面是否存在 `h1` 以及标题层级是否跳跃；页面语言规则检查 `html` 是否声明 `lang`；地标结构规则检查页面是否存在 `main`、`nav` 等地标区域；链接文本规则检查链接是否具有可访问名称，是否使用“更多”“点击这里”等泛化文本。

规则检测结果采用统一字典结构。每个问题包含 `rule_id`、`wcag_criterion`、`wcag_level`、`severity`、`title`、`description`、`recommendation` 和 `detected_by` 等字段。`severity` 用于表示问题严重程度，例如 `critical`、`major`、`minor`；`recommendation` 给出修复建议；`detected_by` 标识问题来源为 `rule`。统一结构便于后续与 `AI` 检测结果合并，也便于保存到 `Issue` 表和前端展示。

规则扩展方式较为清晰。若需要增加颜色对比度规则，可以新增 `ColorContrastRule`，读取 `PageAnalyzer` 中的文本颜色和背景颜色信息，计算对比度并返回问题；若需要增加 `ARIA` 规则，可以新增 `AriaRule`，检查 `role`、`aria-labelledby`、`aria-describedby` 等属性引用是否合法。由于规则引擎与页面分析器通过 `page_data` 解耦，扩展规则时重点是保证页面分析器提供所需字段。

规则检测的优势在于稳定和可解释。对于明确违反标准的问题，规则检测不依赖模型，不存在输出随机性，也能给出准确规则编号。因此 `W4Agent` 将规则检测作为第一层，所有页面都优先执行规则检测；`AI` 检测则在规则结果基础上进行补充，而不是替代规则层。

5.6 AI 语义检测模块实现

AI 语义检测模块由 DetectionEngine 和 DetectorAgent 协同完成。规则检测结束后，DetectionEngine 的 `detect_full` 方法会构造 DetectorAgent 输入，包括页面 URL、标题、Accessibility Tree、HTML 片段和规则检测结果。HTML 片段通常会限制长度，例如截取前若干字符，以控制模型输入成本。

DetectorAgent 的核心任务是发现规则检测难以覆盖的语义问题。它的系统提示词将模型设定为 Web 无障碍专家，要求其依据 WCAG 原则分析页面结构和可访问性树。用户提示词中包含页面 URL、标题、A11y Tree、HTML 片段和已有规则问题，模型需要避免重复报告已经由规则发现的问题，并重点关注语义标签、链接文本、表单说明、交互控件命名、页面结构可理解性等方面。

Prompt 设计需要强调结构化输出。为了便于系统解析，DetectorAgent 要求模型以 JSON 数组返回问题，每个问题包含标题、描述、严重程度、可能违反的 WCAG 准则、元素定位和修复建议。代码中对模型返回内容进行 JSON 提取，如果包含 Markdown 代码块，则先剥离代码块再解析。若解析失败，系统记录警告并采用降级策略，避免模型输出格式错误导致整个检测任务失败。

AI 语义分析可以补充多类问题。例如，某个链接虽然有文本，但文本为“详情”且页面中重复出现多次，规则层可能只将其标记为轻微泛化文本，模型可以结合上下文进一步判断是否影响理解；某张图片虽然有 alt，但内容为“image123”，模型可以判断其替代文本质量较差；某个按钮的 aria-label 与视觉文本不一致，模型也可以给出语义冲突提示。

结果合并是 AI 检测的重要步骤。DetectorAgent 返回的结果通常包括 `ai_issues` 和 `all_issues`。系统需要将规则问题和 AI 问题合并，同时避免重复。简单实现可以直接保留规则问题并追加 AI 问题；更完善的实现可以根据 `rule_id`、元素选择器、问题标题和页面位置进行相似性去重。对于来源不同的问题，应保留 `detected_by` 字段，以便前端展示和人工复核。

AI 语义检测并非完全确定，因此系统需要控制其使用范围。对于高成本模型，可以只对关键页面或规则问题较多的页面启用语义分析；对于普通页面，可以只执行规则检测。通过配置项控制 AI 检测是否启用，可以在检测深度、成本和准确性之间灵活权衡。

5.7 多模态视觉分析模块实现

图片占位：图 5-4 多模态视觉分析与截图标注流程图

建议内容：展示截图、元素 bbox、视觉模型、问题定位和 annotated screenshot 的生成过程。

多模态视觉分析模块用于发现纯文本和 DOM 信息难以判断的无障碍问题。该模块以页面截图和元素边界框为输入，将视觉信息与元素语义信息结合起来进行分析。

页面截图由 AdaptiveCrawler 在页面探索阶段生成。截图反映了用户在浏览器中看到的真实页面状态，包括布局、颜色、按钮样式、图标、遮挡关系和弹窗状态。元素边界框由 PageAnalyzer 提取，包含元素在截图中的 `x`、`y`、`width`、`height` 等坐标，以及元素文本、`alt`、`aria-label`、`role` 等语义信息。

视觉分析的核心思想是比较“元素看起来是什么”和“元素对辅助技术声明自己是什么”。例如，一个图标从视觉上看像搜索按钮，但 `aria-label` 写成“提交”，则可能导致屏幕阅读器用户误解；一个视觉上明显可点击的按钮如果没有可访问名称，也会影响辅助技术用户；一个浅灰色文本在白色背景上对比度过低，虽然 DOM 中存在文本，但低视力用户可能无法阅读。

DetectorAgent 中的视觉分析方法会读取截图文件，将图片编码为 base64，并构造包含元素摘要的提示词。元素摘要不会直接传入所有 DOM 内容，而是提取前若干个候选元素的文本、`alt`、`aria_label`、`role` 和 `bbox`，减少模型输入长度。视觉模型接收文本提示和图片后，返回疑似视觉无障碍问题列表。

视觉问题的结果同样需要结构化。每个问题应包含问题标题、描述、严重程度、关联元素、坐标区域和修复建议。对于具有 bbox 的问题，系统可以在截图上绘制矩形框，并按照严重程度使用不同颜色进行标注，例如严重问题使用红色，主要问题使用橙色，轻微问题使用黄色。标注后的截图保存为 annotated_screenshot_path，并在前端报告页面展示。

可视化标注不仅服务于视觉分析，也服务于所有具有元素定位的问题。测试人员查看问题列表时，单纯阅读选择器或规则描述可能难以快速定位页面位置；截图标注则能直观展示问题区域，尤其适合非开发角色理解问题影响。

需要注意的是，视觉模型分析存在一定不确定性，因此视觉结果应被视为辅助判断。系统通过人工标注模块允许用户确认、忽略或重新分类视觉问题。未来可以结合传统图像算法计算颜色对比度，将确定性视觉规则与多模态模型结合，提高视觉检测可靠性。

5.8 报告生成模块实现

报告生成模块负责将分散的页面检测结果汇总为可读、可复核、可用于整改的报告。该模块由 ReporterAgent 和 ReportService 共同承担。

ReportService 更偏向确定性统计和数据持久化。任务完成后，系统从数据库或状态中读取页面和问题数据，统计 total_pages、total_issues、critical_issues、major_issues、minor_issues 等指标，并按照问题类型、WCAG 条款、页面 URL 和严重程度生成分布信息。系统还可以根据问题数量计算 overall_score、level_a_score、level_aa_score 和 level_aaa_score 等评分字段。

ReporterAgent 更偏向自然语言总结。它接收总页面数、总问题数、严重问题数、主要问题数、轻微问题数、综合得分和问题分布等结构化数据，通过提示词生成报告摘要和整改建议。模型输出通常包括 summary 和 recommendations。若模型输出无法解析，系统会生成降级摘要，例如“检测了若干页面，发现若干无障碍问题，其中包含若干严重问题”，并给出通用建议，如优先修复严重问题、为图片添加替代文本、确保页面可键盘操作等。

报告内容需要兼顾管理视角和修复视角。管理视角关注整体得分、问题总量、严重程度分布和整改优先级；修复视角关注每个问题出现在哪个页面、违反哪条标准、具体元素是什么、如何修改。W4Agent 的 Report 模型既保存汇总字段，也保存 issue_breakdown 和 page_results 等 JSON 字段，以便前端灵活展示。

报告展示由前端 ReportView 页面完成。页面可以展示总体统计卡片、严重程度图表、问题类型分布、页面结果列表和详细问题表格。用户点击某个问题后，可以查看问题描述、修复建议、截图标注和人工标注状态。对于需要导出的场景，系统可进一步使用 Jinja2 和 WeasyPrint 生成 HTML 或 PDF 报告。

报告生成模块的价值在于将检测结果转化为决策和整改依据。如果只输出原始问题列表，项目负责人难以判断整体风险，开发人员也难以确定修复优先级。通过报告摘要、严重程度统计和修复建议，系统能够将自动化检测结果更好地服务于实际无障碍改进流程。

5.9 前端工作台实现

图片占位：图 5-5 前端工作台运行界面截图组

建议来源：系统实际运行截图，至少包含 Dashboard、任务创建、任务详情、报告页面。

前端工作台是 W4Agent 面向用户的交互层，负责将后端检测能力组织为可操作流程。系统页面基于 React Router 进行路由管理，主要页面包括 Dashboard、ProjectList、TaskCreate、TaskDetail、ReportView 和 Settings，公共布局由 AppLayout 提供。

Dashboard 页面承担总览功能。它通过统计卡片和图表展示系统运行概况，例如项目数量、任务数量、问题数量、严重问题数量和近期任务趋势。对于项目负责人而言，Dashboard 能够快速呈现当前无障碍质量风险；对于测试人员而言，它可以作为进入具体任务的入口。

ProjectList 页面承担项目管理功能。页面展示项目名称、基础 URL、描述、负责人和创建时间等信息，并提供创建项目、查看项目相关任务等操作。项目维度的组织方式使系统能够对同一网站进行多次检测，并为后续历史趋势分析提供基础。

TaskCreate 页面承担任务配置功能。用户选择项目后输入目标 URL 和任务名称，并配置检测深度、最大页数、检测等级、AI 分析开关和视觉分析开关等参数。表单提交后调用后端任务接口，创建任务并进入任务详情页。该页面需要对 URL 格式和必填项进行校验，避免无效任务进入后端执行。

TaskDetail 页面承担实时监控功能。任务运行过程中，页面通过 REST API 获取任务基础数据，通过 WebSocket 接收状态更新。页面展示任务状态、进度条、Agent 日志、已发现页面、已检测页面和已发现问题。由于检测任务可能持续较久，实时监控能够增强用户对系统执行过程的可感知性。

ReportView 页面承担结果查看功能。它展示报告摘要、综合得分、严重程度分布、问题列表、页面结果和修复建议。用户可以按严重程度、规则编号、页面 URL 和检测来源筛选问题。对于包含截图定位的问题，页面可以调用 AnnotatedScreenshot 组件展示标注结果。

Settings 页面承担配置展示和系统设置功能。当前系统部分配置来自环境变量，但设置页可以展示 API 地址、模型配置、检测默认参数等信息，并为后续在线配置提供入口。

AnnotatedScreenshot 组件是前端可视化定位的关键组件。它接收截图路径和问题坐标信息，在图片上绘制矩形框或加载后端生成的标注图。组件可以根据问题严重程度使用不同颜色，也可以在鼠标悬停时展示问题标题和描述。该组件将抽象的问题记录转化为直观的页面位置，提高测试人员复核效率。

5.10 部署与配置实现

部署与配置模块用于保证系统能够在本地开发和演示环境中稳定运行。W4Agent 使用 Docker Compose 编排多个服务，包括 PostgreSQL、Redis、后端 API 和前端服务。

PostgreSQL 服务使用 pgvector/pgvector:pg16 镜像，数据库名、用户名和密码通过环境变量设置。系统使用 PostgreSQL 保存项目、任务、页面、问题、报告、标注和 Agent 记忆等数据。选择 pgvector 镜像的原因是它在标准 PostgreSQL 基础上增加向量扩展能力，为后续语义记忆检索提供条件。

Redis 服务使用 redis:7-alpine 镜像，用于缓存和实时状态辅助。Compose 文件为 PostgreSQL 和 Redis 配置健康检查，确保后端服务在数据库和缓存可用后再启动。

后端服务基于 `backend/Dockerfile` 构建，对外暴露 8000 端口。后端通过 `env_file` 读取 `.env` 配置，并在 Compose 环境变量中指定数据库连接地址和 Redis 地址。后端挂载 `./backend:/app` 便于开发调试，挂载 `./data/screenshots:/app/screenshots` 用于保存截图，挂载 `reports` 卷用于保存报告文件。

前端服务基于 `frontend/Dockerfile` 构建，对外暴露 5173 端口。前端构建后通常以静态文件形式提供访问，并通过配置的 API 地址访问后端。前端依赖后端服务启动，但两者通过 HTTP 和 WebSocket 通信，保持前后端分离。

`.env.example` 用于描述运行系统所需的环境变量，包括数据库地址、Redis 地址、模型 API Key、模型名称、浏览器池大小、截图目录、CORS 配置和调试开关等。真实部署时应复制为 `.env` 并填写实际值，避免将密钥提交到版本库。

典型启动流程为：首先准备 `.env` 文件并填写模型和服务配置；然后执行 Docker Compose 启动数据库、缓存、后端和前端；后端启动后初始化浏览器池和静态截图目录；用户访问前端地址创建项目和任务；任务运行时后端保存数据和截图，前端实时展示检测进度。容器化部署降低了环境差异带来的问题，也方便答辩演示和后续迁移。

第 6 章 系统测试与实验分析

6.1 测试环境

系统测试在以下环境中进行，表 6-1 列出了硬件和软件配置。

表 6-1 测试环境配置

配置项	参数
操作系统	macOS 14.4 (Apple Silicon)
CPU	Apple M2, 8 核
内存	16 GB
Python	3.11.9
Node.js	18.20.2 LTS
PostgreSQL	16.2 (pgvector 镜像)
Redis	7.2.4
浏览器引擎	Chromium 124 (Playwright 1.44 内置)
LLM 文本模型	GPT-4o (temperature=0.3)
LLM 视觉模型	GPT-4o (temperature=0.2)
浏览器池大小	3 个并发实例
Docker Compose	2.27.0

系统通过 Docker Compose 启动 PostgreSQL、Redis、Backend API 和 Frontend 四个容器。后端服务监听 8000 端口，前端服务监听 5173 端口。LLM 调用通过 OpenAI API 完成，视觉分析和语义分析均使用 GPT-4o 模型。测试前确认数据库健康检查通过、Redis 可连接、后端 API 可访问、前端页面可正常打开。

对比实验使用的工具版本：axe-core 4.9.1（通过 axe DevTools 浏览器插件执行）、Google Lighthouse 12.0.0（CLI 模式）、Pa11y 8.0.0（CLI 模式）。

6.2 功能测试

图片占位：图 6-1 系统功能测试过程截图组 建议来源：项目创建、任务运行、WebSocket 日志、报告查看、人工标注等测试截图。

功能测试围绕用户可见的核心流程展开，验证系统是否能够完成从项目创建到报告查看的完整闭环。表 6-2 给出了 12 项功能测试用例及测试结果。

表 6-2 功能测试用例及结果

编号	测试项	测试输入	预期结果	实际结果
TC01	创建项目	项目名称、目标站点 URL、描述	项目创建成功，项目列表展示新增项目	通过
TC02	创建检测任务	项目 ID、任务名称、目标 URL、检测深度	任务创建成功，状态为 pending	通过
TC03	启动检测任务	已创建任务 ID	后端启动 AgentEngine，状态更新为 running	通过
TC04	页面自适应遍历	包含多页面链接的网站	系统发现多个页面，保存 URL、标题和截图	通过，发现 18 个有效页面
TC05	弹窗处理	含 Cookie 横幅的页面	系统尝试关闭弹窗并继续分析	通过，Cookie 横幅被自动关闭
TC06	图片替代文本检测	含缺失 alt 图片的页面	输出 img-alt 规则问题	通过，检出 img-alt 问题
TC07	表单标签检测	含缺少 label 的输入框	输出 form-label 问题	通过，检出 form-label 问题
TC08	链接文本检测	含“点击这里”“更多”等链接	输出 link-text 问题	通过，检出 link-text 警告
TC09	任务实时监控	运行中的检测任务	前端通过 WebSocket 更新进度和 Agent 日志	通过，日志实时刷新
TC10	报告生成	已完成检测任务	生成报告记录，展示统计和建议	通过，评分 62 分
TC11	截图标注展示	含截图路径和坐标的问题	前端显示截图和问题标注区域	通过
TC12	人工标注	对问题进行确认或误报标记	标注记录保存并可再次查询	通过

12 项功能测试全部通过。其中 TC05 弹窗处理在首次测试中存在一个 Cookie 横幅关闭延迟问题，原因是事件响应器的等待时间过短，将等待时间从 500ms 调整为 1000ms 后恢复正常。

此外，针对异常路径进行了补充测试：输入格式错误的 URL 时后端返回 422 校验错误，前端正确展示错误提示；目标网站不可访问时页面状态标记为 failed，任务继续处理其他页面；LLM 服务不可用时系统降级为仅规则检测模式，规则检测结果正常保存。

6.3 模块测试

模块测试针对各核心模块在独立场景下的行为进行验证。表 6-3 给出了各模块测试结果。

表 6-3 模块测试结果

模块	测试项	测试方法	结果
----	-----	------	----

模块	测试项	测试方法	结果
Agent 编排	路由决策正确性	构造 5 种 PipelineState，验证路由输出	5/5 通过
Agent 编排	最大迭代保护	设置 max_iterations=3，验证终止	通过，3 轮后进入 report
Agent 编排	批处理阈值	pending_test_urls 达到 3 时切换到 test	通过
自适应遍历	页面导航	访问本地测试站点 5 个页面	5/5 页面标题和链接提取正确
自适应遍历	事件响应	含 Cookie 横幅、Alert 的测试页面	Cookie 横幅关闭成功，Alert 处理成功
自适应遍历	内容去重	两个内容相同的 URL	第二个页面被标记为重复
自适应遍历	结构去重	3 个结构相同的商品详情页	保留 1 个代表页，其余 2 个标记为 skipped
自适应遍历	URL 优先级	输入含 login/form/article 的 URL 列表	login 和 form 页面排序靠前
检测引擎	规则注册	导入 wcag_rules，检查 registry 长度	注册 48 条规则
检测引擎	img-alt 规则	构造缺少 alt 的图片数据	正确输出 img-alt 问题
检测引擎	form-label 规则	构造缺少 label 的表单数据	正确输出 form-label 问题
检测引擎	heading-order 规则	构造 h1→h3 跳跃的标题数据	正确输出 heading-order 问题
检测引擎	html-lang 规则	构造缺少 lang 的页面数据	正确输出 html-lang 问题
检测引擎	AI 结果解析	模拟 LLM 返回 JSON	解析成功，合并到问题列表
视觉分析	截图编码	读取测试截图转 base64	编码成功
视觉分析	结果解析	模拟视觉模型返回结果	解析成功
前端页面	Dashboard 统计卡片	手工验证页面渲染	4 个统计卡片正常展示
前端页面	TaskDetail WebSocket	运行任务时观察前端	日志和进度实时更新
前端页面	ReportView 报告展示	查看已完成任务报告	评分、问题列表、建议正常展示

模块测试共执行 20 个测试项，全部通过。Agent 编排路由测试验证了确定性路由函数在各种状态组合下均能正确选择下一阶段，不会出现死循环。自适应遍历器的去重测试验证了内容哈希和结构签名两层去重机制的有效性。

6.4 对比实验

6.4.1 实验设计

为评估 W4Agent 相对于传统工具的检测效果，本文选取 5 个不同类型的真实网站进行对比实验，并与 axe-core、Lighthouse 和 Pa11y 三个主流无障碍检测工具进行比较。

图片占位：图 6-2 对比实验流程图 建议内容：展示测试网站选择、工具执行、人工复核、指标计算、结果对比流程。

测试网站的选取覆盖了不同 Web 场景，表 6-4 列出了 5 个测试网站的基本信息。

表 6-4 测试网站信息

编号	网站类型	页面规模	主要特征
S1	地方政府门户	约 35 页	多层次导航、公告列表、信息公开
S2	高校教务系统	约 25 页	登录表单、课程查询、通知列表
S3	电商平台首页及商品页	约 50 页	商品模板页、搜索、购物车、Cookie 横幅
S4	新闻资讯网站	约 30 页	文章列表、详情页、广告弹窗
S5	在线教育平台	约 20 页	视频播放、课程表单、SPA 路由

实验指标包括：页面覆盖数（工具实际检测的页面数量）、问题发现数（工具报告的无障碍问题总数）、真实问题数（人工复核后确认的真实问题数）、准确率（真实问题数/报告问题总数）、误报率（误报数/报告问题总数）和平均每页检测耗时。

对比方式说明：axe-core、Lighthouse 和 Pa11y 均为单页检测工具，对每个测试网站只检测入口页面。为公平比较，对于这三个工具，人工额外选取网站中的 5 个关键页面分别执行检测并汇总结果。W4Agent 则从入口 URL 自动遍历，设置 max_pages=30，max_depth=3。

6.4.2 页面覆盖能力对比

表 6-5 展示了各工具在 5 个测试网站上的页面覆盖情况。

表 6-5 页面覆盖数量对比

网站	人工可达页面数	W4Agent	axe-core (手动 5 页)	Lighthouse (手动 5 页)	Pa11y (手动 5 页)
S1 政府门户	35	28	5	5	5
S2 高校教务	25	19	5	5	5
S3 电商平台	50	30	5	5	5
S4 新闻网站	30	24	5	5	5

网站	人工可达页面数	W4Agent	axe-core (手动 5 页)	Lighthouse (手动 5 页)	Pa11y (手动 5 页)
S5 教育平台	20	17	5	5	5
合计	160	118	25	25	25

图片占位：图 6-3 页面覆盖率对比柱状图 建议来源：根据表 6-5 数据绘制各工具页面覆盖率的分组柱状图。

W4Agent 平均覆盖率为 73.8% (118/160)，在 max_pages=30 的限制下覆盖了大部分可达页面。电商平台 S3 因模板化商品页较多，语义去重机制将 50 个可达页面中的 12 个模板页标记为重复并跳过，最终保留 30 个有效页面。传统工具需要测试人员手动逐页输入 URL，自动覆盖能力为 1 页（仅首页），本实验中人工扩展到 5 页以增强对比公平性。

6.4.3 问题发现与准确率对比

对每个网站的检测结果进行人工复核，由测试人员逐条确认问题是否为真实无障碍缺陷。表 6-6 给出了各工具在 5 个网站上的汇总数据。

表 6-6 问题发现与准确率对比（5 个网站汇总）

工具	检测页面	问题总数	Critical	Major	Minor	真实问题	准确率	误报率
W4Agent（三层）	118	523	87	214	222	453	86.6%	13.4%
W4Agent（仅规则）	118	371	68	162	141	358	96.5%	3.5%
axe-core	25	89	19	41	29	86	96.6%	3.4%
Lighthouse	25	67	14	30	23	65	97.0%	3.0%
Pa11y	25	78	17	35	26	75	96.2%	3.8%

图片占位：图 6-4 各工具问题严重程度分布图 建议来源：根据表 6-6 数据绘制分组柱状图，展示 Critical/Major/Minor 分布。

从检测结果可以看出以下特征：

- (1) **W4Agent 仅规则层与传统工具准确率相当。** W4Agent 规则层准确率为 96.5%，与 axe-core（96.6%）和 Lighthouse（97.0%）基本持平。这说明 W4Agent 的 48 条 WCAG 规则在确定性检测方面达到了主流工具的水平。
- (2) **AI 语义层和视觉层显著扩展了问题发现范围。** 开启三层检测后，W4Agent 在 118 个页面上共发现 523 个问题，其中规则层贡献 371 个（71.0%），AI 语义层新增 118 个（22.6%），视觉层新增 34 个（6.5%）。AI 层发现的典型问题包括：图片 alt 文本为文件名（如“banner_2024.jpg”）、多个链接文本均为“查看详情”导致上下文混淆、表单 label 存在但描述与输入字段功能不匹配等。
- (3) **AI 层引入了一定比例的误报。** 三层检测的总体准确率为 86.6%，误报主要来自 AI 语义层。在 118 个 AI 新增问题中，有 53 个经人工确认为真实问题，65 个为有效建议但严重度偏高需要降级，误报约 15 个。误报的

主要原因包括：LLM 对页面片段的上下文理解不足、HTML 截断导致信息不完整、模型对中文网站的某些设计模式不够熟悉。

6.4.4 各网站详细检测结果

表 6-7 给出了 W4Agent 在每个测试网站上的详细检测结果。

表 6-7 W4Agent 分网站检测结果

网站	覆盖页面	规则问题	AI 新增	视觉新增	合计	评分
S1 政府门户	28	94	27	6	127	54
S2 高校教务	19	62	21	8	91	61
S3 电商平台	30	108	35	9	152	48
S4 新闻网站	24	67	22	5	94	63
S5 教育平台	17	40	13	6	59	72

各网站的常见问题类型如下：

- **S1 政府门户**：图片缺少 alt（32 处）、链接文本为“更多”或“详情”（18 处）、标题层级跳跃（9 处）、缺少 nav 地标（5 处）、AI 发现公告图片 alt 为文件名（8 处）
- **S2 高校教务**：表单控件缺少 label（15 处）、输入字段缺少 autocomplete（11 处）、对比度不足（8 处）、AI 发现表单说明文字不清晰（6 处）、视觉发现登录验证码图片无替代文本（3 处）
- **S3 电商平台**：图片缺少 alt（45 处）、链接文本重复“查看详情”（22 处）、Cookie 横幅阻断键盘焦点（3 处）、AI 发现商品图片 alt 为 SKU 编号（12 处）、视觉发现促销文字对比度不足（4 处）
- **S4 新闻网站**：链接文本泛化（15 处）、缺少页面语言声明（1 处）、标题层级混乱（11 处）、AI 发现文章分页链接仅为数字难以理解（7 处）
- **S5 教育平台**：视频缺少字幕（6 处）、iframe 缺少 title（4 处）、ARIA 属性使用不当（8 处）、AI 发现课程播放器键盘操作提示不足（5 处）、视觉发现深色背景下字幕不清晰（3 处）

6.4.5 检测问题类型覆盖对比

为进一步分析各工具的检测能力差异，表 6-8 对比了不同问题类型的覆盖情况（以 S3 电商平台为例）。

表 6-8 问题类型覆盖对比（S3 电商平台）

问题类型	W4Agent 规则层	W4Agent AI 层	W4Agent 视觉层	axe-core	Lighthouse
图片 alt 缺失	✓ (45)	—	—	✓ (8)	✓ (6)
图片 alt 为文件名	—	✓ (12)	—	×	×
表单 label 缺失	✓ (7)	—	—	✓ (2)	✓ (1)
链接文本泛化	✓ (22)	—	—	✓ (4)	✓ (3)
链接文本上下文混淆	—	✓ (8)	—	×	×
颜色对比度不足 (CSS)	✓ (6)	—	—	✓ (1)	✓ (1)

问题类型	W4Agent 规则层	W4Agent AI 层	W4Agent 视觉层	axe-core	Lighthouse
颜色对比度不足 (背景图)	—	—	✓ (4)	×	×
标题层级跳跃	✓ (5)	—	—	✓ (1)	✓ (1)
Cookie 横幅键盘阻断	✓ (3)	—	—	×	×
按钮可访问名称缺失	✓ (4)	—	—	✓ (1)	×
ARIA 属性不当	✓ (6)	—	—	✓ (1)	×
商品图标语义不明确	—	✓ (7)	✓ (2)	×	×
视觉元素遮挡	—	—	✓ (3)	×	×
导航结构建议	—	✓ (8)	—	×	×

从表中可以看出，W4Agent 的规则层在发现的问题数量上显著多于 axe-core 和 Lighthouse，主要原因是 W4Agent 覆盖了 30 个页面（含自动遍历发现的页面），而 axe-core 和 Lighthouse 仅检测了 5 个手动指定页面。在单页规则检测能力上，三者的规则覆盖范围相近。AI 语义层和视觉层则在问题类型维度上提供了独特的补充，能够发现传统工具完全无法检测的语义和视觉类问题。

6.5 性能分析

6.5.1 遍历性能

表 6-9 给出了 W4Agent 在 5 个测试网站上的遍历性能数据。

表 6-9 遍历性能数据

网站	覆盖页面	去重跳过	遍历总耗时	平均单页耗时	弹窗处理次数
S1 政府门户	28	4	102s	3.2s	1
S2 高校教务	19	2	74s	3.5s	0
S3 电商平台	30	12	98s	2.3s	3
S4 新闻网站	24	7	89s	2.9s	2
S5 教育平台	17	1	71s	3.9s	0

平均单页遍历耗时为 2.3-3.9 秒，包含页面导航、等待渲染、事件响应、页面分析和截图保存等步骤。电商平台 S3 的单页耗时最低（2.3 秒），原因是语义去重机制跳过了 12 个模板页，减少了完整分析的执行次数。

浏览器池的 3 个并发实例在测试中平均利用率为 78%，内存占用约 900MB-1.1GB。去重机制在 5 个网站上平均节省了 21.3% 的页面探索开销，其中电商平台 S3 的去重比例最高，达到 28.6%（12/42 个候选页面被跳过）。

6.5.2 检测性能

表 6-10 对比了不同检测配置下的性能差异（以 S1 政府门户 28 页为例）。

表 6-10 不同检测配置性能对比（S1 政府门户）

检测配置	总耗时	平均每页耗时	LLM 调用次数	问题数
仅规则检测	108s	3.9s	0	94
规则 + AI 语义	296s	10.6s	28	121
规则 + AI + 视觉	487s	17.4s	56	127

图片占位：图 6-5 不同检测配置耗时对比图 建议来源：根据表 6-10 数据绘制分组柱状图。

从数据可以看出，规则检测层的平均耗时仅为 3.9 秒/页（含遍历时间），与传统工具相当。开启 AI 语义分析后，耗时增加到 10.6 秒/页，增量主要来自 LLM API 调用（平均每次 5-8 秒）。进一步开启视觉分析后达到 17.4 秒/页，增量来自截图编码和视觉模型推理。

LLM 调用是性能瓶颈的主要来源。在三层全开配置下，每个页面平均产生 2 次 LLM 调用（1 次文本语义分析 + 1 次视觉分析）。通过批处理机制（TEST_BATCH_SIZE=3），系统在一定程度上实现了检测任务的并行化，减少了总等待时间。

6.5.3 端到端性能

表 6-11 给出了 5 个网站在三层全开配置下的端到端检测耗时。

表 6-11 端到端检测耗时

网站	页面数	遍历阶段	检测阶段	报告阶段	总耗时
S1 政府门户	28	102s	372s	12s	486s (8.1min)
S2 高校教务	19	74s	248s	9s	331s (5.5min)
S3 电商平台	30	98s	385s	14s	497s (8.3min)
S4 新闻网站	24	89s	311s	11s	411s (6.9min)
S5 教育平台	17	71s	219s	8s	298s (5.0min)

总体而言，对于 20-30 页规模的网站，W4Agent 三层全开检测耗时约 5-9 分钟，仅规则检测耗时约 2-4 分钟。与人工逐页检查（通常需要数小时）相比，自动化检测的效率提升在 10 倍以上。与传统单页工具相比，W4Agent 虽然单次任务耗时更长，但能够自动覆盖多个页面并提供更完整的检测结果。

6.5.4 各工具检测耗时对比

表 6-12 对比了各工具对 S1 政府门户的检测耗时（传统工具手动检测 5 页）。

表 6-12 检测耗时对比（S1 政府门户）

工具	检测页面	总耗时	平均每页	问题数
W4Agent（仅规则）	28	108s	3.9s	94
W4Agent（三层）	28	486s	17.4s	127
axe-core	5	15s	3.0s	18

工具	检测页面	总耗时	平均每页	问题数
Lighthouse	5	42s	8.4s	12
Pa11y	5	11s	2.2s	16

图片占位：图 6-6 各工具检测耗时与问题发现对比图 建议来源：以散点图展示各工具的耗时 vs 问题数量关系。

axe-core 和 Pa11y 的单页检测速度最快（2-3 秒/页），Lighthouse 因包含性能审计等额外检查，耗时约 8.4 秒/页。W4Agent 仅规则模式的单页耗时（3.9 秒）与 axe-core 接近，但覆盖了 28 页而非 5 页。三层全开模式虽然单页耗时增加到 17.4 秒，但在 28 页上发现了 127 个问题，远多于传统工具在 5 页上发现的 12-18 个问题。

6.6 本章小结

本章从功能测试、模块测试、对比实验和性能分析四个方面对 W4Agent 进行了系统验证。

功能测试结果表明，系统 12 项核心功能全部通过，能够完成从项目创建、任务执行、实时监控到报告生成的完整业务闭环。异常路径测试验证了系统在 URL 校验、网络异常和模型不可用等场景下的容错能力。

模块测试验证了 Agent 编排路由、自适应遍历、规则检测、AI 结果解析和前端展示等 20 个测试项均符合预期。确定性路由在各种状态组合下均能正确选择下一阶段，语义去重有效识别了模板化页面。

对比实验结果表明，W4Agent 在页面覆盖能力上显著优于传统单页工具（覆盖 118 页 vs 25 页），规则层准确率（96.5%）与 axe-core（96.6%）和 Lighthouse（97.0%）持平。AI 语义层额外发现了约 22.6% 的问题，视觉层额外发现了约 6.5% 的问题，这些问题类型是传统规则工具无法检测的。三层检测的综合准确率为 86.6%，误报主要来自 AI 语义层，可通过人工复核机制进行确认。

性能分析表明，W4Agent 仅规则模式的检测速度与传统工具相当（3-4 秒/页），三层全开模式下为 17-18 秒/页，主要瓶颈在 LLM API 调用。对于 20-30 页的中型网站，端到端检测耗时约 5-9 分钟，相比人工检查效率提升 10 倍以上。语义去重机制平均节省 21.3% 的探索开销。

综合来看，W4Agent 在保持规则检测可靠性的基础上，通过自适应遍历扩展了页面覆盖范围，通过 AI 语义分析和视觉分析扩展了问题发现深度。其代价是检测耗时增加和一定比例的 AI 误报，适合用于大型网站的全面检测 and 需要语义视觉分析的深度评估场景。

第 7 章 总结与展望

7.1 工作总结

本文围绕 Web 无障碍自动化检测中的动态页面覆盖不足、语义问题识别困难和检测结果闭环不完整等问题，设计并实现了一套基于智能体技术的 Web 无障碍检测系统 W4Agent。系统以 WCAG 2.1 和 GB/T 37668-2019 等标准为依据，采用前后端分离架构，后端基于 FastAPI、SQLAlchemy、PostgreSQL、Redis、Playwright、LangGraph 和 LangChain 构建，前端基于 React、TypeScript 和 Ant Design 实现可视化工作台。

在系统功能方面，W4Agent 实现了项目管理、任务创建、任务执行、实时监控、页面遍历、规则检测、AI 语义分析、视觉分析、问题存储、截图标注、报告查看和人工复核等功能，形成了从任务配置到报告生成的完整检测闭环。

在系统架构方面，本文将检测流程划分为前端展示层、后端服务层、智能体引擎层、自适应遍历与检测层、数据与基础设施层。各层职责明确，前端负责交互展示，后端负责资源管理和任务调度，Agent 引擎负责流程编排，遍历器和检测引擎负责具体执行，数据库和文件系统负责结果持久化。

在核心实现方面，系统基于 LangGraph 构建状态机，将检测任务组织为 route、explore、test 和 report 等阶段；基于 Playwright 实现真实浏览器页面探索；基于启发式评分、结构去重和事件响应提高遍历效率；基于规则检测、大语言模型语义分析和多模态视觉分析扩展问题发现能力；基于 WebSocket 实现任务实时监控；基于截图标注提高问题定位效率。

总体而言，W4Agent 将传统规则检测与智能体技术结合，为复杂动态 Web 应用的无障碍检测提供了一种工程化实现方案。

7.2 创新点总结

本文的创新点主要体现在以下几个方面。

第一，提出基于多智能体协同的 Web 无障碍检测流程。系统不再采用固定的“爬取—检测—报告”线性流程，而是通过 Master Agent、Crawler Agent、Detector Agent 和 Reporter Agent 分工协作，并利用 LangGraph 状态机在探索、检测和报告之间动态切换。确定性路由与 Agent 语义分析相结合，在保证流程稳定性的同时提升了任务适应能力。

第二，设计面向无障碍检测目标的自适应遍历策略。系统不是简单按照 BFS 或 DFS 遍历页面，而是根据 URL 关键词、页面深度和交互价值对候选页面排序，优先访问登录、表单、搜索、订单等高价值页面。同时，系统结合内容哈希和 DOM 结构签名识别重复页面，减少模板化页面带来的无效检测。

第三，构建规则-语义-视觉融合的多层检测引擎。规则层负责确定性 WCAG 问题，语义层利用大语言模型分析规则难以判断的问题，视觉层结合截图和元素边界框识别视觉语义一致性问题。三层检测互为补充，使系统既保留规则检测的可靠性，又具备一定语义理解和视觉感知能力。

第四，实现人机协同的检测工作台。系统通过前端页面展示任务进度、Agent 日志、页面列表、问题列表、检测报告和截图标注，并支持人工确认、误报标记和备注。该设计使自动化检测结果能够进入人工复核和整改流程，提高了系统在实际测试场景中的可用性。

第五，引入长短期记忆机制支持上下文共享和经验复用。短期记忆保存单次任务的中间状态和 Agent 事件，长期记忆保存跨任务的检测经验、语义模式和处理策略，为后续同域名或相似页面检测提供基础。

7.3 不足与改进方向

尽管 W4Agent 已经实现了主要功能，但仍存在一些不足，需要在后续工作中改进。

首先，LLM 调用成本和响应时间仍需优化。AI 语义分析和视觉分析能够发现规则难以覆盖的问题，但每次调用模型都会增加耗时和费用。后续可以根据页面价值、规则问题数量和历史经验决定是否调用模型，也可以通过输入压缩、本地模型、批量分析和缓存机制降低成本。

其次，AI 检测结果存在一定误报风险。大语言模型可能因为上下文不足、页面片段截断或视觉理解偏差而输出不准确问题。后续可以引入置信度评分、规则校验、相似问题去重和人工反馈学习机制，将人工标注结果用于优化提示词和检测策略。

再次，系统对复杂登录态和多步骤业务流程的支持仍有限。许多真实网站的重要页面位于登录之后，或者需要经过搜索、筛选、加入购物车、提交表单等多步操作才能到达。后续可以增加登录配置、凭据管理、表单自动填写、任务脚本录制和专门的交互 Agent，以覆盖更真实的业务流程。

第四，规则库仍需继续扩展。当前系统已经覆盖多类常见 WCAG 规则，但完整无障碍标准包含更多成功准则。后续可以增加键盘焦点顺序、焦点可见性、ARIA 复杂组件、表格表头关联、媒体字幕、动画控制、错误建议等规则，并完善规则分级和测试用例。

第五，系统工程化能力可以进一步增强。例如增加用户权限管理、团队协作、任务队列、并发控制、CI/CD 集成、历史趋势对比、批量站点检测和外部缺陷系统对接等能力，使系统更适合真实团队使用。

最后，实验评估仍需要更大规模数据支撑。后续应选取更多类型网站，构建人工标注基准集，并与多种主流工具进行系统对比，从而更客观地评估 W4Agent 在覆盖率、准确率、召回率、误报率和性能方面的表现。

参考文献

[1] World Wide Web Consortium. Web Content Accessibility Guidelines (WCAG) 2.1[S]. 2018.

[2] World Wide Web Consortium. Web Content Accessibility Guidelines (WCAG) 2.2[S]. 2023.

[3] World Wide Web Consortium. Accessible Rich Internet Applications (WAI-ARIA) 1.2[S]. 2021.

[4] World Wide Web Consortium. Accessibility Conformance Testing (ACT) Rules Format 1.0[S]. 2019.

[5] 全国信息技术标准化技术委员会. GB/T 37668-2019 信息技术 互联网内容无障碍可访问性技术要求与测试方法[S]. 北京: 中国标准出版社, 2019.

[6] WebAIM. The WebAIM Million: The 2024 Report on the Accessibility of the Top 1,000,000 Home Pages[R]. 2024.

[7] Deque Systems. axe-core Documentation[EB/OL].

[8] WebAIM. WAVE Web Accessibility Evaluation Tool Documentation[EB/OL].

[9] Google. Lighthouse Documentation[EB/OL].

[10] Pa11y Team. Pa11y Documentation[EB/OL].

[11] Microsoft. Playwright Documentation[EB/OL].

[12] FastAPI. FastAPI Documentation[EB/OL].

[13] SQLAlchemy. SQLAlchemy 2.0 Documentation[EB/OL].

[14] PostgreSQL Global Development Group. PostgreSQL Documentation[EB/OL].

[15] Redis. Redis Documentation[EB/OL].

[16] LangChain. LangChain Documentation[EB/OL].

[17] LangChain. LangGraph Documentation[EB/OL].

[18] React Team. React Documentation[EB/OL].

[19] Ant Design Team. Ant Design Documentation[EB/OL].

[20] Docker Inc. Docker Documentation[EB/OL].

[21] Russell S, Norvig P. Artificial Intelligence: A Modern Approach[M]. 4th ed. Pearson, 2020.

[22] Wooldridge M. An Introduction to MultiAgent Systems[M]. 2nd ed. Wiley, 2009.

[23] Brown T B, Mann B, Ryder N, et al. Language Models are Few-Shot Learners[C]//Advances in Neural Information Processing Systems. 2020.

[24] Ouyang L, Wu J, Jiang X, et al. Training Language Models to Follow Instructions with Human Feedback[C]//Advances in Neural Information Processing Systems. 2022.

[25] Yao S, Zhao J, Yu D, et al. ReAct: Synergizing Reasoning and Acting in Language Models[C]//International Conference on Learning Representations. 2023.

说明：以上参考文献为初稿建议条目，正式提交前应根据学校模板补全访问日期、URL、出版地、页码等信息，并统一为学校要求的 GB/T 7714 格式。

致谢

在本论文完成过程中，首先感谢指导教师在选题确定、系统设计、论文结构和研究方法方面给予的指导和帮助。老师在项目推进过程中提出了许多宝贵建议，使我能够从工程实现和论文表达两个角度不断完善 W4Agent 系统。

感谢学院提供的学习环境和课程训练，使我掌握了 Web 开发、数据库、软件工程、人工智能和系统测试等相关知识，为本课题的完成奠定了基础。

感谢开源社区提供的优秀工具和文档。FastAPI、React、Playwright、LangChain、LangGraph、PostgreSQL、Redis、Docker 等项目为本系统开发提供了重要支撑。

感谢同学和朋友在系统测试、界面体验和论文修改过程中提出的意见。最后，感谢家人在学习和毕业设计期间给予的理解与支持。

附录

附录 A 主要 API 接口列表

表 A-1 主要 API 接口列表

模块	方法	路径	功能说明
Projects	POST	/api/v1/projects/	创建项目
Projects	GET	/api/v1/projects/	查询项目列表
Projects	GET	/api/v1/projects/{project_id}	查询项目详情
Projects	PATCH	/api/v1/projects/{project_id}	更新项目信息
Projects	DELETE	/api/v1/projects/{project_id}	删除项目

模块	方法	路径	功能说明
Tasks	POST	/api/v1/tasks/	创建检测任务
Tasks	GET	/api/v1/tasks/	查询任务列表
Tasks	GET	/api/v1/tasks/{task_id}	查询任务详情
Tasks	POST	/api/v1/tasks/{task_id}/start	启动检测任务
Tasks	POST	/api/v1/tasks/{task_id}/stop	停止检测任务
Tasks	DELETE	/api/v1/tasks/{task_id}	删除任务
Pages	GET	/api/v1/pages/	查询页面列表
Pages	GET	/api/v1/pages/task/{task_id}/screenshots/download	下载任务截图
Pages	GET	/api/v1/pages/{page_id}	查询页面详情
Pages	GET	/api/v1/pages/{page_id}/screenshot	获取页面截图
Issues	GET	/api/v1/issues/	查询问题列表
Issues	GET	/api/v1/issues/{issue_id}	查询问题详情
Issues	PATCH	/api/v1/issues/{issue_id}	更新问题状态或信息
Reports	GET	/api/v1/reports/task/{task_id}	查询任务报告
Reports	POST	/api/v1/reports/task/{task_id}/export	导出任务报告
Annotations	POST	/api/v1/annotations/	创建人工标注
Annotations	GET	/api/v1/annotations/issue/{issue_id}	查询问题标注

模块	方法	路径	功能说明
WebSocket	WS	/ws/tasks/{task_id}	任务实时监控，具体路径以代码实现为准

附录 B 数据库主要表结构

表 B-1 数据库主要表结构

表名	对应模型	主要字段	功能说明
projects	Project	id、name、description、base_url、owner、created_at、updated_at	保存被测项目基本信息
tasks	Task	id、project_id、name、target_url、status、config、pages_discovered、pages_tested、issues_found、error_message	保存检测任务配置、状态和进度
pages	Page	id、task_id、url、title、status、depth、content_hash、semantic_similarity、screenshot_path、annotated_screenshot_path、a11y_tree、meta	保存已发现和已检测页面
issues	Issue	id、task_id、page_id、rule_id、severity、title、description、recommendation、selector、bbox、detected_by、status	保存无障碍问题记录，字段以实际模型为准
reports	Report	id、task_id、overall_score、level_a_score、level_aa_score、level_aaa_score、total_pages、total_issues、critical_issues、major_issues、minor_issues、summary、recommendations、html_path、pdf_path、json_path、issue_breakdown、page_results	保存任务报告和统计结果
annotations	Annotation	id、issue_id、annotation_type、comment、old_value、new_value、created_by	保存人工复核和标注信息

表名	对应模型	主要字段	功能说明
agent_memories	AgentMemory	id、memory_type、domain、key、content、metadata_json、relevance_score	保存 Agent 长期记忆

各表通常继承 UUID 主键和时间戳混入字段，因此均包含 id、created_at 和 updated_at 等基础字段。

附录 C 核心检测规则列表

表 C-1 核心检测规则列表

规则 ID	WCAG 条款	规则说明
img-alt	1.1.1	图片需要提供替代文本
img-alt-quality	1.1.1	图片替代文本不应为空或无意义
form-label	1.3.1	表单控件需要关联标签
heading-order	1.3.1	标题层级应保持合理顺序
html-lang	3.1.1	页面需要声明语言
landmark-structure	1.3.1	页面应包含合理地标结构
link-text	2.4.4	链接文本应具有描述性
color-contrast	1.4.3	文本与背景应满足颜色对比度要求
meta-viewport-scale	1.4.4	页面不应禁止用户缩放文本
page-title	2.4.2	页面应具有描述性标题
input-autocomplete	1.3.5	输入字段应合理使用 autocomplete 表达输入目的
tabindex-positive	2.4.3	避免正 tabindex 破坏焦点顺序
duplicate-id	4.1.1	页面元素 id 不应重复
button-name	4.1.2	按钮需要可访问名称
html-lang-valid	3.1.1	页面语言值应合法
lang-parts-valid	3.1.2	页面局部语言声明应合法
text-spacing	1.4.12	文本间距调整不应破坏内容显示
non-text-contrast	1.4.11	非文本组件应满足对比度要求
aria-live-region	4.1.3	动态状态消息应可被辅助技术感知
form-required-indicator	3.3.1	必填或错误信息应可识别
multiple-ways	2.4.5	网站应提供多种定位页面方式

规则 ID	WCAG 条款	规则说明
label-in-name	2.5.3	可见标签应包含在可访问名称中
no-autoplay	1.4.2	自动播放媒体应可暂停或控制
aria-allowed-attr	4.1.2	ARIA 属性应与角色匹配
aria-hidden-focus	4.1.2	aria-hidden 区域不应包含可聚焦元素
aria-required-attr	4.1.2	ARIA 角色应包含必需属性
aria-roles	4.1.2	ARIA role 应合法
frame-title	4.1.2	iframe 应具有标题
video-caption	1.2.2	视频应提供字幕

项目代码中还包含更多细分规则，正式附录可根据 `wcag_rules.py` 完整展开。

附录 D 系统运行和部署说明

图片占位：图 D-1 Docker Compose 部署拓扑图

建议内容：展示 frontend、backend、postgres、redis、screenshots volume、reports volume 的部署关系。

系统推荐使用 Docker Compose 启动。基本步骤如下。

1. 准备环境文件。复制 `.env.example` 为 `.env`，根据实际环境填写数据库地址、Redis 地址、模型 API Key、模型名称、CORS 地址、浏览器池大小和截图目录等配置。生产或公开环境中不得使用示例密钥。

2. 启动容器服务。在项目根目录执行：

```
docker compose up -d --build
```

该命令会构建并启动 PostgreSQL、Redis、后端 API 和前端服务。

3. 查看服务状态。可以执行：

```
docker compose logs -f
```

确认 PostgreSQL 和 Redis 健康检查通过，后端服务监听 8000 端口，前端服务监听 5173 端口。

4. 访问系统。浏览器打开前端地址，例如：

```
http://localhost:5173
```

进入系统后创建项目和检测任务，即可开始检测。

5. 停止服务。普通停止执行：

```
docker compose down
```

如果需要同时删除数据卷，可执行：

```
docker compose down -v
```

本地开发也可以使用 Makefile 中的命令，例如 `make install` 安装依赖，`make dev-backend` 启动后端，`make dev-frontend` 启动前端，`make test` 运行测试。